



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 94. La elaboración de prototipos en el desarrollo de
sistemas. Diseño de interfaces de aplicaciones**

AUTOR: Mariam Dobón Muñoz

Actualización 2017

ÍNDICE

1	LA ELABORACIÓN DE PROTOTIPOS EN EL DESARROLLO DE SISTEMAS.....	4
1.1	INTRODUCCIÓN.....	4
1.2	CONCEPTOS DE PROTOTIPOS	4
1.2.1	DEFINICIÓN	4
1.2.2	OBJETIVO Y BENEFICIOS	5
1.2.3	COSTE.....	6
1.3	TIPOS DE PROTOTIPOS.....	7
1.3.1	PROTOTIPOS SEGÚN LA FUNCIÓN	7
1.3.2	PROTOTIPOS SEGÚN LA FUNCIONALIDAD IMPLEMENTADA	7
1.3.3	PROTOTIPOS SEGÚN EL DESTINO.....	10
1.4	CUÁNDO USAR PROTOTIPADO	11
1.5	VENTAJAS Y DESVENTAJAS DE LOS PROTOTIPOS	11
1.6	MODELOS DE CICLO DE VIDA RELACIONADOS CON EL PROTOTIPADO.....	12
1.6.1	MODELO ITERATIVO DE PROTOTIPADO	12
1.6.2	MODELO EN ESPIRAL	14
1.7	HERRAMIENTAS DE PROTOTIPADO	15
1.7.1	TÉCNICAS DE CUARTA GENERACIÓN	15
1.7.2	COMPONENTES DE SOFTWARE REUTILIZABLES	16
1.7.3	ESPECIFICACIONES FORMALES Y ENTORNOS PARA PROTOTIPOS.....	16
1.8	EL PROTOTIPADO EN MÉTRICA VERSIÓN 3	16
1.8.1	INTERFAZ DE GESTIÓN DE PROYECTOS (GPI)	16
1.8.2	ANÁLISIS DEL SISTEMA DE INFORMACIÓN (ASI)	16
1.8.3	DISEÑO DEL SISTEMA DE INFORMACIÓN (DSI).....	17
1.8.4	LA TÉCNICA DEL PROTOTIPADO EN MÉTRICA V3.....	17
2	DISEÑO DE INTERFACES DE APLICACIONES.	19
2.1	INTRODUCCIÓN.....	19
2.2	CONCEPTOS DE INTERFACES DE USUARIO	19
2.2.1	TIPOS DE INTERFACES DE USUARIO.....	20
2.2.1.1	SEGÚN SU CONSTRUCCIÓN	20
2.2.1.2	SEGÚN LA FORMA DE INTERACCIÓN DEL USUARIO	20
2.3	DISEÑO DE INTERFACES DE USUARIO	24
2.3.1	PRINCIPIOS DE DISEÑO DE INTERFACES DE USUARIO	25
2.3.2	ENFOQUES DE DISEÑO UI, UX, IxD.....	26
2.4	USABILIDAD.....	27
2.5	HERRAMIENTAS Y LENGUAJES PARA EL DESARROLLO DE INTERFACES DE USUARIO	28
2.6	GUÍAS DE ESTILO	29
2.6.1	COMERCIALES	29

2.6.2	CORPORATIVAS	30
2.7	ESTÁNDARES RELACIONADOS	30
3	CONCLUSIÓN	32
4	GLOSARIO	33

1 La elaboración de prototipos en el desarrollo de sistemas.

1.1 Introducción

Cuanto más grande es un sistema, y más participantes colaboran en su desarrollo, mayor es la probabilidad de que el desarrollo no satisfaga completamente las necesidades del cliente. Las razones que llevan a dicha insatisfacción, la cual muchas veces se convierte para el usuario en frustración, son múltiples, variando desde problemas de entendimiento humano hasta cuestiones culturales, pasando por visiones de un mismo aspecto desde diferentes puntos de vista.

El software es un producto un tanto especial, muy flexible, pero a la vez complejo de construir. Es esta flexibilidad la que permite a un ordenador hacer “casi cualquier cosa”, pero a la vez es la que también hace que sea muy fácil para el equipo de desarrollo desviarse de las necesidades del usuario y construir un sistema que difiera con lo que éste espera.

Existen técnicas de análisis de requisitos que se han desarrollado para facilitar este entendimiento entre desarrolladores y usuarios. Una de estas técnicas se basa en la creación de prototipos, que al igual que los prototipos en otras disciplinas de las ingenierías, son versiones preliminares de un producto antes de ponerse en producción, con objeto de poder estudiar su comportamiento, deficiencias, etc. y poderlos subsanar y mejorar.

En el caso del software, un prototipo se suele crear para poder estudiar en más detalle una parte del sistema. Las razones son múltiples y pueden ir desde mostrar al usuario la interfaz de usuario, para que sea fácil y rápida de manejar, hasta cuestiones internas de desarrollo como para probar el rendimiento de un determinado algoritmo.

El coste de hacer un prototipo suele ser elevado, pues requiere tiempo y esfuerzo. Entonces ¿por qué se justifica dicho esfuerzo? La respuesta es clara: se hace un prototipo para estudiar el comportamiento del sistema, ya sea ante el usuario, ante ciertos tipos de entrada o ante ciertos procesos internos. Corregir los fallos una vez que esté en producción el sistema será mucho más caro que si lo hacemos en las primeras etapas del análisis. No obstante, para realizar un prototipo de forma rentable se deben dar ciertas condiciones, entre las que están las de disponer de herramientas que permitan construirlo de manera rápida y eficiente.

A lo largo de este epígrafe se revisará el concepto del prototipado, que no es más que una herramienta que permite, de una forma rápida, implementar y estudiar el funcionamiento de una parte de un sistema, para alcanzar un entendimiento común entre desarrolladores y usuarios.

1.2 Conceptos de prototipos

1.2.1 Definición

Se puede definir un **prototipo** como un producto software que se obtiene en una fase anterior a la finalización de un proyecto de desarrollo, con objeto de permitir evaluar, modificar especificaciones de usuario, demostrar al usuario en qué punto del camino se encuentra, etc.

Al igual que en ingeniería aeronáutica se construye un prototipo de un avión para poder ver en el túnel de viento el comportamiento del avión, en el caso del software, el prototipo se puede poner a funcionar en un entorno de pruebas para que los usuarios puedan experimentar con él, realizando su actividad diaria, para determinar si el prototipo responde a sus necesidades.

No obstante, un prototipo no tiene porqué implementar todas las funcionalidades del sistema. Normalmente se usan para implementar una parte de las mismas o un aspecto que no queda claro. Así, por ejemplo, un prototipo podría ser la interfaz de usuario del sistema en un sistema de control del tráfico aéreo.

Así, un prototipo es un producto más, generado durante el desarrollo del sistema, que permite facilitar la comunicación entre usuarios y desarrolladores. En muchos casos, los usuarios no tienen una formación técnica, lo que lleva a que tengan dificultades para visualizar el funcionamiento del sistema cuando éste es aún un conjunto de documentos que especifican la funcionalidad que debe ofrecer. Para un usuario es mucho más sencillo estudiar un prototipo, algo que es tangible, que responde a sus acciones, que un documento especificando todo lo que el sistema debe satisfacer. El usuario no sabe exactamente qué es lo que quiere, pero normalmente sí que es capaz de determinar qué es lo que necesita si lo ve.

Una vez se ha usado un prototipo, puede servir de base para el desarrollo del sistema o ser desechado completamente para comenzar el desarrollo del sistema desde el comienzo. En caso de que se use como base, deberá estar construido de manera suficientemente robusta y puede ir evolucionando hasta convertirlo en el sistema final, o también se puede utilizar el conocimiento adquirido en el desarrollo para el desarrollo del sistema final. En cambio, si el prototipo está diseñado para ver un aspecto muy específico del sistema, ha sufrido muchas modificaciones estructurales, o simplemente está desarrollado como una maqueta, sin funcionalidad por detrás, el prototipo se descarta.

1.2.2 Objetivo y beneficios

Según Sommerville, un prototipo permite demostrar conceptos, probar opciones de diseño, y en general, informarse más sobre el problema y sus posibles soluciones. Un prototipo permite reducir el riesgo en un proyecto de desarrollo de software mediante la reducción de la incertidumbre, mejorando la comunicación entre los diferentes participantes. Esta reducción se logra mediante una comunicación fluida entre usuarios y desarrolladores ya que el prototipo es un elemento tangible, que realiza una determinada funcionalidad, y que interactúa con el usuario.

La interacción usuario-prototipo debe ser adecuadamente evaluada por el usuario para poder generar una retroalimentación a los desarrolladores. Del resultado de la experiencia que el usuario tenga con el prototipo se obtendrá la confirmación para seguir adelante con el proyecto, o si la experiencia ha sido negativa, una serie de modificaciones a realizar sobre la visión de los desarrolladores.

En cualquiera de los casos, la retroalimentación obtenida por la creación del prototipo es muy valiosa, ya sea confirmatoria, indicando que se va por el buen camino, como correctora, indicando las áreas en que debe trabajarse más. De la claridad que se tenga de los requisitos del sistema una vez realizada la evaluación se tomará la opción de modificar el prototipo existente e incorporarle los nuevos requisitos, o si todos los requisitos ya están claros seguir adelante y construir el sistema definitivo.

En muchas ocasiones, un prototipo se suele usar para entender una parte del sistema que no se entiende completamente, de la que se tiene dudas o no se sabe cómo va a responder. Otros usos de los prototipos, además de facilitar la comunicación usuario-desarrollador, pueden ser:

- Poder probar el comportamiento de una funcionalidad concreta, como un algoritmo, una comunicación con un módulo ya existente.
- Para poder probar una nueva tecnología que no se sabe muy bien cómo responderá. Cuestiones como el uso de una nueva interfaz hombre-máquina, como por ejemplo la basada en una pantalla táctil o un guante de datos (Dataglove), o un desarrollo sobre un nuevo sistema operativo, pueden, si no son convenientemente validadas, llevar al fracaso del proyecto.
- Permiten ver las fortalezas y debilidades del software que se va a construir. Si se aprovecha bien la retroalimentación de las evaluaciones del prototipo es posible corregir las desviaciones en las primeras fases del desarrollo.

Gordon y Bieman realizaron un estudio sobre 39 proyectos de desarrollo de software que usaron la técnica del prototipado, y observaron una serie de mejoras respecto a otros proyectos, como son:

- **Mejora de la usabilidad del sistema.** El sistema era más fácil de usar, debido a que el prototipo había sido evaluado por el usuario previamente y las dificultades de uso se habían eliminado en las primeras fases del proyecto.

- **Mejor concordancia entre el sistema y las necesidades del usuario.** Al existir una comunicación desarrolladores-usuarios a través del prototipo, los errores de entendimiento de los desarrolladores se redujeron y los requisitos plasmaban con mayor fidelidad las necesidades de los usuarios.
- **Mejora de la calidad del diseño.** El diseño del sistema final se basaba en el conocimiento adquirido en el desarrollo del prototipo, redundando en un incremento de la calidad de producto final.
- **Mejora en el mantenimiento.** Al estar mejor diseñado el software y parecerse más a lo que el usuario quiere, se reducen o eliminan muchas tareas de mantenimiento. Además, al ser un mejor diseño, es más fácil realizar las modificaciones.
- **Reducción del esfuerzo de desarrollo.** Al tener una base de la que partir, el prototipo, el esfuerzo en el desarrollo del software se reduce. Se pueden reutilizar diseños de pantallas, algoritmos, estructura de la aplicación, etc. El conocimiento previo adquirido al desarrollar el prototipo ayuda también a agilizar el desarrollo del sistema final.

1.2.3 Coste

Es lógico pensar que, si hay más tareas a realizar en el proyecto, como las correspondientes a la construcción del prototipo y evaluación del mismo, el coste del proyecto pueda ser superior a un desarrollo siguiendo un modelo tradicional sin prototipado.

Como se ha visto en el apartado anterior, el cometido principal del prototipo es mejorar la comunicación entre usuarios y desarrolladores, o incrementar el conocimiento en ciertos puntos que no están claros. Este esfuerzo adicional en la construcción del prototipo no debe verse como una carga adicional que hay que realizar en todos los proyectos, sino más bien como una herramienta alternativa al proceso tradicional de establecimiento y obtención de requisitos del sistema. De hecho, no en todos los proyectos es adecuado utilizar prototipado, sino solamente cuando hay razones que lo justifican.

En los casos en que sea necesario realizar un prototipo, el esfuerzo adicional para confeccionarlo debe verse como una inversión en conocimiento que simplificará las fases posteriores del proyecto, no siendo siempre cierto que el coste total final del proyecto se incremente.

Si el usuario experimenta con el prototipo, hace una evaluación, y los comentarios se incorporan a los requerimientos del sistema en fases tempranas, se elimina mucho trabajo de construcción y prueba al acertar las necesidades del usuario a la primera. Sin prototipado, todo el esfuerzo que se habrá dedicado a probar todas las funcionalidades y que hay que volver a probar si hay cambios considerables en el sistema puede ser muy costoso. Usando prototipos, las pruebas son mucho más simples y rápidas, y permiten identificar de forma temprana discrepancias entre los desarrolladores y usuarios. Así se eliminan muchos costes asociados a rehacer el software ya construido, debido normalmente a cambios solicitados por los usuarios en las últimas fases del proyecto o a una mala educación de requisitos en las primeras fases de desarrollo.

Además, la aprobación confirmatoria del usuario a la hora de validar los requisitos permite reducir la incertidumbre del proyecto y lograr una planificación y una estimación de costes mucho más ajustada y real. Se tiene una mayor certeza en que lo que se está construyendo es lo que el usuario quiere.

Como se ha comentado, el conocimiento adquirido durante la construcción del prototipo, ya sea sobre su construcción o sobre partes del prototipo que se reutilizan en el sistema final, es de gran utilidad para el desarrollo del sistema final. También este conocimiento simplifica el desarrollo del sistema final y reduce el esfuerzo total.

No obstante, hay que tener en cuenta que para asegurar el éxito de los prototipos es esencial que se disponga de herramientas para generarlos rápidamente, de forma que los costes sean controlados y los usuarios puedan usar el sistema desde las primeras etapas del desarrollo del software.

1.3 Tipos de prototipos

Los prototipos se pueden clasificar desde diferentes perspectivas. A continuación, se muestran algunas de estas clasificaciones ofrecidas por diferentes autores.

1.3.1 Prototipos según la función

Según Whitten, los prototipos se pueden clasificar en las siguientes cuatro clases:

De viabilidad

Estos prototipos se usan para probar la viabilidad de una determinada tecnología aplicable al sistema de información. Sobre los resultados obtenidos con el prototipo se toma una decisión de si seguir adelante con la tecnología o por el contrario dar marcha atrás y buscar otra más viable, pues tecnologías poco maduras o inadecuadas para un proyecto en particular pueden hacer fracasar el mismo.

De necesidad

Estos prototipos, también denominados de descubrimiento, se usan para descubrir las necesidades de los usuarios con respecto a la empresa. Los usuarios muchas veces no saben exactamente lo que quieren o tienen dificultades para definirlo, por lo que suele ser de utilidad construir un prototipo de este tipo. El prototipo se comporta, de cara al usuario, como el sistema real. Esto le permite experimentar como si del sistema real se tratase, pero centrándose en ver las necesidades que no están claras a un coste mucho menor que el sistema final.

Evaluando la experiencia del usuario con el prototipo se permite obtener nuevos requerimientos del sistema, modificar los ya existentes o incluso eliminar alguno. No es raro que un proceso que el usuario pensase que se podía hacer de una determinada manera al usar el prototipo se dé cuenta que es mejor realizarlo de otra, o incluso que es excesivamente complicado o requiere mucho esfuerzo. El prototipo es una herramienta muy valiosa para poder identificar las verdaderas necesidades del usuario.

De diseño

Estos prototipos, también conocidos como prototipos de comportamiento, se usan para simular el diseño del sistema de información final. Estos prototipos se centran en el funcionamiento del sistema, y no en el aspecto, como lo hacen los de necesidad. Con estos prototipos, el analista espera que el usuario evalúe su funcionamiento, facilidad de uso y aprendizaje, aspecto de pantallas y navegación, y procedimiento de uso.

De implantación

Los prototipos de implantación, también conocidos como prototipos de producción, son la evolución de los prototipos de diseño hacia un sistema que está cerca de ser el sistema final. Estos prototipos normalmente no incluyen funcionalidades de edición, control de errores, seguridad o mensajes. Estas funcionalidades se pueden incorporar de una forma relativamente sencilla si se decide evolucionarlo hacia el sistema definitivo.

Estos prototipos tomaron mucha importancia con la aparición de los *Lenguajes de Cuarta Generación* y las herramientas de generación automática de código, ya que permitían modificar las características de la aplicación, con las nuevas recomendaciones del usuario y volver a generar la aplicación. Además, estas herramientas ofrecen muchos medios para definir qué hay que hacer en la aplicación y la herramienta se encarga de cómo hacerlo, con lo que se reduce sustancialmente el número de líneas de código.

1.3.2 Prototipos según la funcionalidad implementada

Generalmente, se habla de prototipos y maquetas como conceptos equivalentes y se utilizan términos como *mockup*, *wireframe* y *sketch*, especialmente en el ámbito del desarrollo web, para referirse a distintos tipos de prototipos, cayendo en el error de tratarlos indistintamente o de usarlos como sinónimos de prototipo, cuando su definición y uso realmente son diferentes.

Sketch

Un *sketch* es un dibujo rápido o bosquejo de guía general que no tiene por qué tener muchos detalles y que reproduce un concepto, idea, o generalidad de un proyecto de una manera muy sencilla. Por lo general, se realiza a mano con lápiz y borrador por su facilidad en el momento de plasmar una base o punto de partida. El entregable es una imagen o un papel.

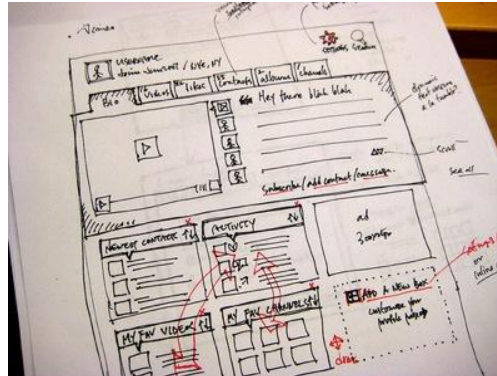


Figura 1 Sketch

Wireframe

Un *wireframe* es una representación básica y de baja calidad del desarrollo de una página web o el diseño de una interfaz, cuya finalidad es mostrar al cliente un diseño o boceto rápido para facilitar la comunicación cliente-desarrollador. Su principal objetivo es proporcionar una visualización del diseño o las funcionalidades de una web o una app para poder definir los requisitos en una etapa muy temprana del proyecto. Se realizan para facilitar la definición de los principales grupos de contenido, la estructura de la información, y la descripción y visualización de la interfaz. El entregable es una imagen.

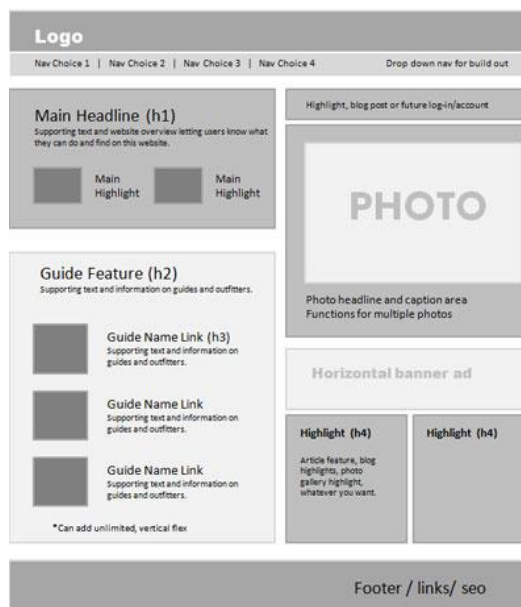


Figura 2 Wireframe

Maqueta

Las *maquetas* son construcciones que implementan solamente la interfaz de usuario. No tienen funcionalidad por detrás, sino que muestran los elementos de la interfaz y la navegación. De hecho, se construyen para mostrar exclusivamente el funcionamiento de la interfaz de usuario. Solamente se desarrolla código, de forma manual o automática, para lo relacionado con la interfaz de usuario (comportamiento de los elementos de la pantalla, llenado de los elementos de la pantalla con datos

ficticios o incluso estáticos, navegación entre pantallas, etc.), pero no se desarrolla código para la funcionalidad que realiza la interfaz (alta de un cliente, asociar un producto a un cliente, etc.). El entregable suele ser HTML.

Sommerville define un caso particular de maqueta, que es la que denomina Mago de Oz. Al igual que en la película, el sistema que usa este tipo de maqueta solamente muestra la interfaz de usuario. El procesamiento de los datos que realiza la aplicación lo hace realmente una persona. Estas maquetas son útiles para que el usuario pueda ver cómo se comporta el sistema de una forma “casi real” sin necesidad de realizar mucho esfuerzo en el desarrollo del prototipo.

Mockup

Un *mockup* es una maqueta en terminología de diseño gráfico. Representa especialmente un diseño gráfico editorial, para presentar al cliente cómo quedaría su producto, identidad de marca, página web, etc. En el caso de una página web representa la estructura de la información, visualiza el contenido y demuestra las funcionalidades básicas de ésta. Asimismo, permite revisar la parte visual del proyecto. El entregable es una imagen.



Figura 3 Mockup

Prototipo

Un *prototipo*, en el sentido estricto de la palabra, es una representación navegable del producto final. La calidad puede variar entre media y alta. Simula la interfaz de usuario y debe permitir que el usuario experimente interactuando con la interfaz y el contenido del proyecto, pruebe las principales interacciones de una manera similar al producto final. Los prototipos implementan una parte del sistema a construir. No suelen implementar solamente la interfaz de usuario, sino que también implementan las funcionalidades que hay por detrás, unas veces sin validaciones, otras con algunas y otras con todas las validaciones. El entregable es un HTML, PPT, animación o formato navegable.



Figura 4 Prototipo

Así, las maquetas, *wireframes* y *mockups* solamente se centran en mostrar la interfaz de usuario, nunca la funcionalidad o parte de ella. Son útiles sobre todo en las primeras fases del desarrollo de la interfaz, en donde se espera que el usuario dé una confirmación de que la interfaz planteada es la que necesita y no se quiere realizar un gran esfuerzo en el desarrollo de una interfaz que puede que no sea la que el usuario espera. En cambio, los prototipos, sí que suelen incluir parte de la funcionalidad.

1.3.3 Prototipos según el destino

Los prototipos se pueden clasificar, según el destino que se les dé, en prototipos desechables y en prototipos evolutivos. El primer tipo de prototipos, como su nombre indica, se desecha una vez que finaliza su cometido mientras que el segundo tipo se evoluciona del prototipo al sistema final.

Prototipos desechables

Estos prototipos son de usar y tirar. Dentro de esta categoría están los prototipos rápidos y las maquetas, mockups y wireframes, ya comentados anteriormente.

Los prototipos rápidos se construyen rápidamente, para poder evaluarlos cuanto antes por el usuario o el desarrollador. Debido a que en la mayoría de ellos se usan herramientas automáticas de generación de los prototipos, éstos suelen ser grandes, lentos e ineficientes. Por ello, tras su uso se descartan y se construye de nuevo el sistema final.

Estos prototipos desechables, debido al destino que tienen, deben poder construirse con facilidad para evaluarlo en una etapa temprana del desarrollo y, además, han de ser baratos. Su coste nunca debería superar el 10% del coste total del proyecto.

Los objetivos de estos prototipos son:

- Reducir el riesgo de construir un producto que se aleje de las necesidades del usuario.
- Reducir los costes totales de desarrollo al disminuir las correcciones en etapas avanzadas del mismo.
- Aumentar las posibilidades de éxito del proyecto.

Una vez diseñado y construido el prototipo se procede a su experimentación y refinamiento para, a continuación, pasar al diseño y construcción del sistema real.

En algunos casos como, por ejemplo, el diseño de las pantallas con las herramientas visuales que existen actualmente es muy probable que pueda reaprovecharse para el sistema definitivo una vez incorporadas las modificaciones identificadas. El resto, el código y datos incorporados al funcionamiento de la maqueta (que normalmente suele ser muy simple y con muy pocas validaciones

o incluso con datos estáticos) se descartarían para sustituirlos por el verdadero código que incorpora la funcionalidad y validaciones completas.

Prototipos evolutivos

Los prototipos evolutivos son aquellos que evolucionan desde su estadio de prototipo hasta convertirse en la solución final.

La principal ventaja de estos prototipos es que el prototipo construido se aprovecha y no se descarta. Por el contrario, si el prototipo no se ha desarrollado siguiendo rigurosos estándares y con calidad, es muy probable que el sistema final tenga problemas de calidad y de rendimiento.

Es muy importante determinar desde el principio si el prototipo será evolutivo, ya que se deberá hacer mucho más hincapié en aspectos de calidad, rendimiento, modificabilidad, documentación, etc. que si se plantea un prototipo desechable.

1.4 Cuándo usar prototipado

Los prototipos no deben usarse de forma sistemática en un proyecto, sino que debido al incremento de esfuerzo inicial que requieren deben ser usados solamente cuando se dan ciertas condiciones que indican que el uso del prototipado es adecuado a las características del proyecto de desarrollo.

Sommerville considera que el prototipado se debe usar en proyectos pequeños o medianos, o subproyectos dentro de un proyecto grande, o aquellos que tienen poca duración. Otras condiciones que indican la adecuación para el uso de prototipos son:

- Situaciones en las que los requisitos no están claros
- Para probar una funcionalidad que no se sabe muy bien cómo va a responder.
- Para probar algoritmos específicos o ideas innovadoras.
- Para probar una tecnología nueva que se desconoce cómo va a responder.

1.5 Ventajas y desventajas de los prototipos

La técnica del prototipado, al igual que cualquier otra, tiene ciertas ventajas e inconvenientes. Las principales **ventajas** del uso de prototipos son:

- Los usuarios se hacen partícipes del desarrollo, ya que participan de una forma más activa en el mismo evaluando los prototipos, que siempre es mucho más fácil de evaluar que una especificación de requisitos software.
- Los usuarios tienen mucha más facilidad a la hora de describir lo que necesitan si lo ven en un prototipo que si lo tienen que hacer de palabra y sin ningún elemento de referencia.
- Hay una continua revisión de los requisitos del sistema por los usuarios al estar trabajando en la evaluación de los prototipos y redefinición de los requisitos.
- Los prototipos ayudan mejorar la comunicación usuario – desarrollador, con lo que se reduce en gran medida el riesgo de que se apruebe por parte de los usuarios el diseño y se rechace el sistema ya en implantación. Es muy probable que los usuarios tengan una visión diferente del sistema, ellos piensen que el diseño es de una forma y que los desarrolladores lo vean de otra. Mediante el prototipo se ayuda a que ambas partes compartan una visión común. Gracias esta mejora en la comunicación, se reducen los malentendidos entre desarrolladores y usuarios, se pueden detectar las funcionalidades que les faltan, se mejora la identificación de los servicios más confusos, se pueden localizar los requisitos inconsistentes a tiempo, etc.
- Si un prototipo está operativo, aunque de forma reducida, se puede empezar a formar a los usuarios-formadores y preparar la documentación para la formación del grueso de usuarios.

Las **desventajas** del uso de las técnicas de prototipado son las siguientes:

- Se puede forzar a los usuarios a elegir de forma demasiado temprana una única opción de prototipo, descartándose automáticamente el resto de posibles opciones. El analista suele seguir adelante con un prototipo, normalmente el primero que evalúa y suscita interés entre los usuarios, y puede que se dejen de lado opciones que son mejores, pero no han llegado a evaluarse en detalle.
- Los sistemas basados en prototipos suelen ser poco flexibles a cambios posteriores ya que no se han desarrollado pensando en la flexibilidad.
- No siempre es fácil cambiar un prototipo. Si el desarrollo no se ha hecho con la calidad suficiente, puede resultar muy difícil de modificar.
- Muchas veces el cliente, al ver el prototipo, tiende a pensar que es el sistema final y es muy difícil convencerle de que se trata de un prototipo y que el sistema final debe ser desarrollado de nuevo.
- Al trabajar los usuarios con un prototipo que implementa parte de sistema, muchas veces no tienen una visión completa del sistema, sino solamente de la parte que realmente pueden ver y usar.

1.6 Modelos de ciclo de vida relacionados con el prototipado

Una vez que se ha visto qué es un prototipo, el uso que tiene y los diferentes tipos de prototipos que se pueden construir, se va a proceder a ver cómo se integra la técnica de los prototipos en el modelo de ciclo de vida de un desarrollo.

Hay autores que formalizan la construcción de prototipos dentro de un determinado modelo de ciclo de vida. Sin embargo, hay otros autores, como por ejemplo Pressman, que consideran que es factible construir un prototipo usando cualquier modelo de ciclo de vida, basta con incorporar los pasos necesarios para la realización del mismo. Así, si queremos construir un prototipo durante el diseño de una aplicación que se está desarrollando según un modelo de ciclo de vida en cascada, basta con incorporar en la fase de análisis o diseño las etapas de construcción del prototipo. Lo mismo sucede con los modelos de ciclo de vida evolutivos o de desarrollo ágil, donde el prototipo se puede incorporar desde las fases tempranas del desarrollo con prototipado evolutivo, y que finalmente acabará formando parte del sistema final.

Asimismo, hay que destacar el desarrollo de aplicaciones parametrizables ya preconstruidas. Estas aplicaciones son fabricadas por una tercera empresa con una funcionalidad estándar genérica. Hoy en día, muchos proyectos se basan en adaptar a las necesidades de un cliente esta funcionalidad estándar genérica. Sistemas como SAP, Vignette Content Manager, Clarify Amdocs o Remedy son muy comunes en el mundo empresarial. Dado que estos sistemas son en sí ya un sistema completamente operativo, se podrían considerar como un primer prototipo, que a través de la evaluación por parte del usuario pueda ser adaptado a sus necesidades finales. Por supuesto, este sistema no se consideraría como un prototipo desechable, sino que sería un prototipo evolutivo que se adaptaría a los nuevos requerimientos del usuario.

Al igual que los paquetes de aplicaciones, los nuevos paradigmas de desarrollo de software usando reutilización de componentes también contribuyen en gran medida al desarrollo de prototipos, ya que se facilita enormemente gracias a la disponibilidad de componentes ya construidos y probados que se interconectan con la aplicación con muy poco esfuerzo.

1.6.1 Modelo iterativo de prototipado

El modelo que se usa para el prototipado es un modelo iterativo, el cual va pasando por un conjunto de fases con las que se logrará ir perfeccionando, de forma iterativa, el prototipo. Las fases por las que pasa el desarrollo del prototipo son las siguientes:

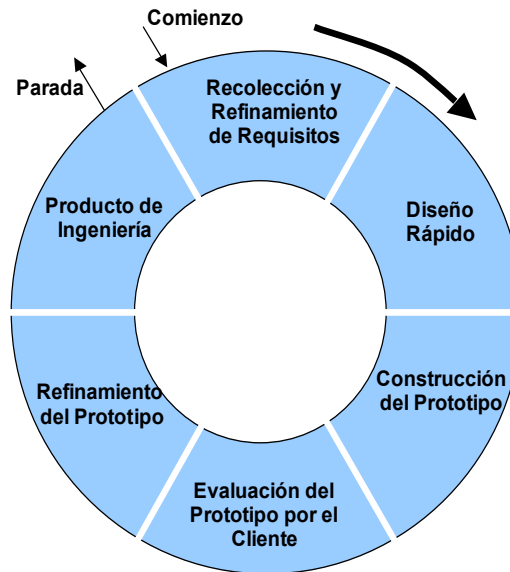


Figura 5 Modelo iterativo de prototipado

Primero se recogen los requisitos conocidos del sistema. Con estos primeros requisitos se construirá el primer prototipo. El resto de requisitos se irá obteniendo a partir de las evaluaciones y del conocimiento que se vaya obteniendo durante cada iteración.

Normalmente, para que el proyecto se haga en poco tiempo se diseña el prototipo rápidamente, que se centrará principalmente en los aspectos que serán visibles para el usuario final. Aspectos como la interfaz de usuario o los informes serán aspectos a tener en cuenta.

Con el diseño se construye el prototipo. Normalmente la calidad del prototipo no es muy buena porque se construye rápido y no se suelen incorporar muchas validaciones, pues el prototipo se construye para ver el comportamiento del sistema en ciertos aspectos.

Una vez construido se le entrega al cliente para que lo evalúe. Éste dará sus comentarios que servirán para añadir nuevos requisitos o modificar los que no se hayan entendido completamente. A partir de la experiencia durante el desarrollo, tanto de los desarrolladores como los usuarios, podrán aparecer nuevos requisitos e identificarse potenciales problemas que deberán ser resueltos.

Así, se refina el prototipo incorporando los nuevos requisitos, tras lo cual se volverá a evaluar por el usuario. Este proceso de refinamiento se realiza iterativamente hasta que el prototipo cumple con los requisitos del usuario y éste da su aceptación.

Cuando el prototipo sea dado por válido por el usuario, por último, se construye el producto definitivo. Este último paso se realiza de dos formas distintas dependiendo si se usa prototipado desechable o evolutivo:

- Si se usa el prototipado evolutivo, el sistema final se construirá a partir de la evolución del prototipo.
- Si se usa prototipado desechable, el sistema final se construirá utilizando un modelo de procesos tradicional, ya que ahora son conocidos todos los requisitos del sistema.

Existe una disquisición entre diferentes autores sobre si el prototipo debe ser evolucionado o descartado. Así, por ejemplo, Connell opina que lo deseable es evolucionar el prototipo hasta convertirlo en el sistema final.

Sin embargo, otros autores como por ejemplo Brooks opinan que un prototipo debe descartarse una vez utilizado. Dice textualmente:

“En la mayoría de los proyectos, el primer sistema construido apenas se utiliza. Tal vez sea muy lento, muy grande o torpe en su uso, o reúna las tres características a la vez. No existe otra opción que comenzar de nuevo, aunque sea doloroso, es lo

mejor, y construir una revisión rediseñada en la que se resuelvan estos problemas... Cuando se aplica un concepto nuevo de sistema o una tecnología nueva, se tiene que construir un sistema inservible y que sea necesario desechar, porque incluso la mejor planificación no es omnisciente como para que el sistema esté perfecto desde la primera vez. Por lo tanto, la pregunta de la administración no es si debe construirse un sistema piloto y desecharlo. Esto tendrá que hacerse. La única pregunta es si se planea la construcción de un desechable o se promete entregárselo a los clientes.”

El cliente debe ser consciente en todo momento, de que, si se está usando un prototipo desechable, éste no será evolucionado hasta el sistema real. Si el prototipo es una maqueta del sistema, de cara al usuario el sistema funciona perfectamente, salvo con los requerimientos nuevos identificados. El problema está en lo que hay detrás, el código que soporta todas las funcionalidades del sistema, que es muy probable que no sea todo lo robusto, fiable y flexible que deba ser, o incluso en algunos casos puede ni llegar a existir por tratarse de una simple maqueta que muestra la navegación y aspecto del sistema.

Hay que tener en cuenta que el desarrollo del prototipo se suele hacer con cierta rapidez, y convencer al cliente, después de ver “algo” operativo, de que construir el sistema final va a costar mucho más que el prototipo, puede ser muy difícil, pues para él, que no conoce lo que hay detrás de la aplicación, le parece igual el prototipo que el sistema definitivo.

Lo mejor para estos casos es dejar claro al principio del proyecto cómo se va a desarrollar éste, que el prototipo será descartado cuando cumpla su cometido y que el sistema definitivo se desarrollará tras el prototipo, partiendo de cero, y con un enfoque de calidad, flexibilidad y robustez.

1.6.2 Modelo en Espiral

El modelo en Espiral, propuesto por Bohem, conjuga la naturaleza iterativa de la construcción de prototipos con los aspectos controlados y sistemáticos del modelo en cascada.

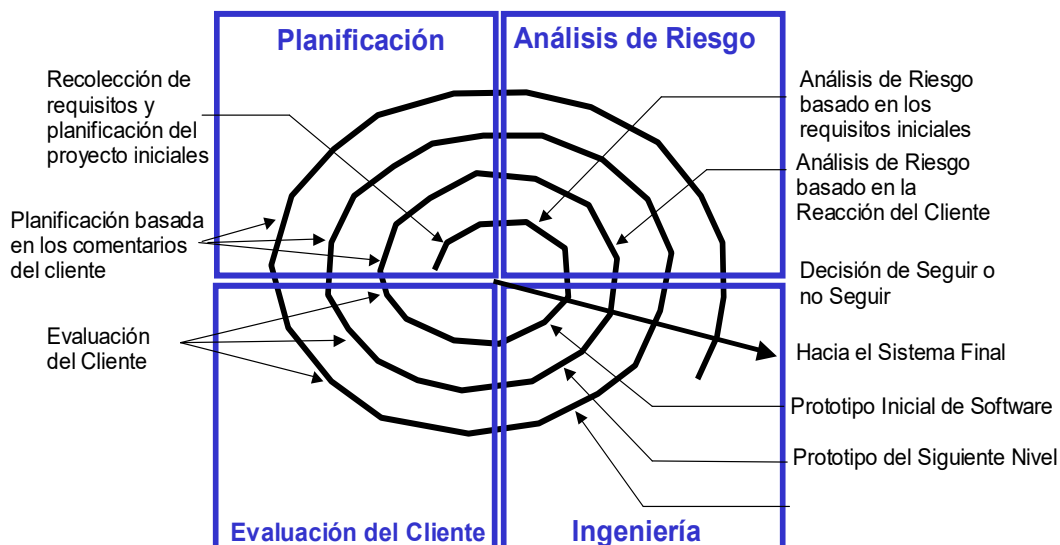


Figura 6 Modelo en espiral

El modelo se basa en una espiral dirigida por el riesgo. Primero se planifica en cada iteración lo que se va a hacer, se analiza el riesgo de hacerlo y se elige la mejor alternativa, se construye aplicando un proceso de ingeniería y por último se evalúa lo que se ha alcanzado. De la evaluación se determina si se debe continuar con otro ciclo de la espiral o por el contrario se debe parar porque se ha completado el sistema.

En el análisis de riesgos se puede plantear la realización de prototipos para evaluar el riesgo de un cierto aspecto que no está claro. Del resultado de la evaluación se tomará una opción u otra a la hora del siguiente paso, el paso de desarrollo o ingeniería.

En cada iteración, en la fase de ingeniería, se construirá, siguiendo el modelo de proceso que se considere oportuno, un elemento o un prototipo del sistema final. En las primeras evoluciones la entrega quizás sea un documento o un prototipo, mientras que en la última será el sistema completo.

Este modelo da cabida a desarrollar prototipos durante el análisis de riesgos, para reducir el riesgo en cualquier aspecto que se haya identificado, ya sea en tema de requisitos, cuestiones técnicas, procesos, etc.

Además, el modelo en espiral también permite un desarrollo basado en un prototipo evolutivo, al que en cada ciclo de la espiral se le van incorporando nuevas funcionalidades al prototipo que se está desarrollando, hasta convertirse este en el sistema final.

1.7 Herramientas de prototipado

Existen en el mercado numerosas herramientas que se pueden usar para desarrollar prototipos. Algunas de ellas son específicamente para ello mientras que otras son herramientas de desarrollo que incorporan facilidades para realizar prototipos o que se pueden aprovechar.

Las plataformas actuales de desarrollo permiten generar diseños de pantallas de forma visual. Los componentes de la pantalla se distribuyen en la misma y se les configuran las características. Algunas herramientas permiten simular el comportamiento de la pantalla y si se incorpora algo de código se permite incluso realizar navegación entre diferentes pantallas.

En muchos casos, sobre todo en las primeras fases del análisis, es muy útil el uso de lápiz y papel para dibujar, delante del usuario, un boceto del aspecto de las pantallas. El lápiz y el papel ofrecen mucha flexibilidad a la hora de poner de acuerdo a ambas partes sin entrar en los detalles. Además, el lápiz y el papel requieren mucho menos esfuerzo a la hora de dibujar y borrar que otras herramientas, por lo que, en las primeras fases, cuando todo son aún ideas, requiere muy poco esfuerzo en dibujar y borrar. Cuando esta primera visión dibujada en papel esté aceptablemente estable se puede proceder a usar herramientas informáticas para crear una versión mucho más fidedigna de las pantallas, que requerirá más esfuerzo que el boceto en papel, pero que no será en vano ya que en el papel ya se llegó a un cierto “acuerdo”.

Como ya se ha visto, para que un prototipo sea efectivo se debe poder construir rápidamente, para que se pueda entregar al usuario y éste pueda hacer las recomendaciones oportunas. En los últimos años, han proliferado las herramientas de prototipado basadas en la web, que permiten diseñar mockups, wireframes y/o prototipos rápidos visuales y no funcionales pero muy útiles para conseguir en poco tiempo un diseño rápido. Por ejemplo, Balsamiq y Moqups son herramientas online para realizar mockups, Invision sirve para crear mockups o prototipos tanto para páginas web como aplicaciones iOS o Android, al igual que la herramienta Marvel. Mockup Builder permite crear prototipos rápidos con más de 800 plantillas disponibles, Proto.io sirve para realizar prototipos de muy buena calidad...etc. Las opciones son numerosas y muy variadas.

Además, los diseñadores web realizan prototipos en HTML no funcionales para las aplicaciones web, que posteriormente pueden ser usados como base para el desarrollo del sistema. Andriole o Tannenbaum identifican los siguientes métodos y herramientas para la creación de prototipos:

- Técnicas de Cuarta Generación
- Componentes de Software Reutilizables
- Especificaciones Formales y Entornos para Prototipos

1.7.1 Técnicas de cuarta generación

Bajo este concepto subyacen numerosas técnicas que van desde lenguajes de consulta e informes a bases de datos hasta lenguajes procedimentales de alto nivel, pasando por generadores de programas y aplicaciones. El punto común de todas estas técnicas es que permiten la creación de software rápidamente.

1.7.2 Componentes de software reutilizables

Si en vez de construir el prototipo desde el principio, línea a línea de código, se construye éste mediante el ensamblado de componentes preconstruidos y ya probados, el tiempo de desarrollo del prototipo se reduce en gran medida. Por supuesto, para poder hacer esto es necesario que los componentes a utilizar se hayan construido con la idea en mente de usarse para la reutilización, sin requerir al programa que lo usa conocer cómo está construido internamente. Para que esta técnica funcione se debe elaborar previamente un catálogo de los componentes existentes, para que la localización de los mismos sea fácil y rápida.

1.7.3 Especificaciones formales y entornos para prototipos

Se han desarrollado múltiples lenguajes formales que permiten especificar sistemas de forma alternativa al lenguaje natural. En la actualidad existen entornos interactivos que facilitan al analista:

- Crear interactivamente una especificación de un sistema o software.
- Que permitan comunicarse con herramientas que generen de forma automática el código ejecutable del sistema a partir de la especificación.
- Permiten al cliente usar el código ejecutable generado para poder refinar los requisitos formales.

1.8 El Prototipado en Métrica versión 3

La Metodología de Desarrollo de Software de la Administración Española, Métrica versión 3, es una metodología flexible que permite elegir entre varios modelos de procesos para el desarrollo, entre los que se encuentra el prototipado. El uso de los prototipos se encuentra integrado en las diferentes fases, actividades y tareas de Métrica. A continuación, se detallan los puntos en los que Métrica hace uso del prototipado.

1.8.1 Interfaz de Gestión de Proyectos (GPI)

Los modelos que tiene contemplados Métrica v3 son:

- **Clásico o en Cascada.**
- **Por Subsistemas.** Se sigue una aproximación incremental. El sistema se divide en subsistemas y cada subsistema se desarrolla siguiendo su propia aproximación, que suele ser incremental.
- **Por Prototipos.** Se crea en las primeras etapas del proyecto un prototipo que posteriormente se va evolucionando hasta incorporar todos los requisitos del sistema. Esta aproximación se suele usar en desarrollos orientados a objetos.
- **Híbrida.** Se realiza una división en subsistemas, que se desarrollan bajo un enfoque diferente de los demás.

Métrica deja al Jefe de Proyecto la tarea de determinar cuál de los modelos de proceso debe aplicarse. En la tarea GPI.2.1, en la Interfaz de Gestión de Proyectos, el Jefe de Proyecto debe elegir la Estrategia de Desarrollo, es decir, qué modelo o modelos de proceso se van a aplicar al proyecto en particular.

1.8.2 Análisis del Sistema de Información (ASI)

Métrica recomienda el uso de prototipado y diseño de diálogos para colaborar con los usuarios en el desarrollo de las interfaces de usuario del sistema y así lograr una mejor comunicación y entendimiento entre ambas partes. Estas técnicas permiten al usuario participar activamente en el

desarrollo y que éste se vaya familiarizando con el sistema y aporte su conocimiento a la construcción y perfeccionamiento del mismo.

En la actividad *ASI 8 Definición de Interfaces de Usuario*, se usa la técnica de prototipado para crear las interfaces de Usuario y de Impresión. En esta actividad, se realizan las siguientes tareas:

- *ASI 8.3. Especificación de los Formatos Individuales de la Interfaz Pantalla*
- *ASI 8.4. Especificación del Comportamiento Dinámico de la Interfaz*
- *ASI 8.5. Especificación de los Formatos de Impresión*

En la actividad *ASI 9 Análisis de Consistencia y Especificación de Requisitos* se recomienda el uso de la técnica del prototipado para validar, en la tarea *ASI 9.3 Validación de los Modelos*, la consistencia de la interfaz de usuario mediante un prototipo que puede ser estático o dinámico.

1.8.3 Diseño del Sistema de Información (DSI)

En la Actividad *DSI 3. Diseño de Casos de Usos Reales*, en la tarea *DSI 3.3 Revisión de la Interfaz de Usuario* se realiza una revisión de la Interfaz de Usuario con el objetivo de realizar un diseño detallado de la misma. Se parte del prototipo que se desarrolló en el análisis y se le incorpora la información nueva que ha ido apareciendo.

También se revisa la interfaz de usuario, la navegación entre ventanas, los elementos que forman cada interfaz, sus características (que deben ser consistentes con los atributos con los que están relacionadas), su disposición, y cómo se gestionan los eventos relacionados con los objetos.

En la actividad *DSI 5. Diseño de la Arquitectura de Módulos del Sistema*, en la tarea *DSI 5.3 Revisión de la Interfaz de Usuario*, se revisa la descomposición funcional en diálogos según la arquitectura modular. Se realizan las modificaciones oportunas teniendo en cuenta los requisitos de rendimiento, seguridad y características de cada diálogo.

También se revisa la navegación entre ventanas, la información necesaria para cada diálogo, si es obligatoria u opcional, las dependencias entre datos, los rangos de valores y los valores predefinidos.

Tras terminar esta tarea, si ha habido cambios sustanciales en el diseño de la interfaz, se recomienda validarlos por el usuario, dejando de forma opcional incorporar dichos cambios al prototipo para que sean más fácilmente validados por el usuario.

1.8.4 La Técnica del Prototipado en Métrica v3

Métrica v3 considera como técnica, en su guía de técnicas, la técnica de prototipado. Se centra principalmente en el aspecto visual de las pantallas e informes para lograr una interfaz eficiente y compatible con las necesidades de los usuarios. Cuando los usuarios se sienten a gusto con el aspecto del sistema, éstos lo usan de una forma más eficazmente, cometiendo menos errores y mejorando por tanto la productividad.

Para la construcción de los prototipos, Métrica recomienda estudiar los siguientes aspectos:

- Quiénes son los usuarios a los que va dirigido el prototipo. Se deben determinar los diferentes perfiles de usuarios que usan el sistema.
- Qué funciones realizan dichos usuarios. Se definen las funciones y se relacionan dichas funciones con los perfiles de usuarios identificados previamente.

Se deben tener en cuenta los siguientes aspectos:

- Utilizar conceptos, términos y símbolos familiares al usuario, de modo que sea fácil de aprender y comprender.
- Mantener la coherencia dentro del propio sistema y entre sistemas:
 - Proporcionando abreviaturas estándar.
 - Aplicando las mismas reglas de interacción a través de toda la interfaz.

- Utilizando el mismo formato para los mensajes de error, de aviso o advertencia, mandatos, títulos y comandos con significado similar.

De esta manera, la experiencia y los conocimientos adquiridos por los usuarios en sistemas previos puede generalizarse a otros sistemas, reduciendo los costes y recursos necesarios para la formación.

- Facilitar la exploración del sistema sin riesgo, permitiendo interrumpir y deshacer las acciones realizadas. De esta forma, el usuario puede utilizar todas las funcionalidades del sistema y trabajar de forma más rápida y eficiente, con la seguridad de que cualquier error puede rectificarse.
- Dificultar la selección de acciones destructivas y no reversibles, pidiéndole verificación de cualquier acción que conlleve un riesgo importante.
- Proporcionar información sobre el estado de ejecución de las funciones, es decir, si la función invocada está en proceso, si se ha completado satisfactoriamente o se ha producido algún error. El usuario nunca debería preguntarse si el sistema está ocupado procesando una transacción o está esperando una nueva entrada y si el sistema no responde a tiempo, iniciar nuevas tareas con el convencimiento de que la anterior finalizó de forma satisfactoria.
- Agrupar las funciones de forma lógica y presentar primero las más utilizadas.
- Buscar la eficiencia en el diálogo evitando cambios frecuentes entre los dispositivos de entrada, tales como el ratón y el teclado.

Estos principios se aplicarán, en mayor o menor medida, en función de las características del entorno tecnológico y de las necesidades planteadas. No obstante, la adopción de una guía de estilos facilita la coordinación en el equipo de desarrollo y potencia la reutilización.

Tras considerar estos aspectos hay que realizar un análisis de la información a mostrar en cada una de las pantallas. Se considerarán los siguientes puntos:

- Identificar los diferentes tipos de información como, por ejemplo, campos de datos, títulos, comandos y mensajes de error, con el fin de organizar la pantalla en áreas específicas y conseguir un equilibrio, regularidad, secuencialidad, así como, una simetría en la composición de las mismas.
- Estudiar el espacio disponible en las pantallas para determinar qué datos y en qué situación deben aparecer en las mismas, utilizando un formato de visualización que permita al usuario una rápida asimilación de la información.
- Intentar agrupar los datos relacionados y mostrar solamente aquellos que son esenciales para la ejecución de la función o para la toma de una decisión, eliminando todas las entradas que sean innecesarias. Nunca se debe pedir al usuario que introduzca información que pueda adquirirse automáticamente o calcularse internamente.
- Mantener la coherencia entre la entrada y la visualización de datos.
- Proteger al usuario de intentar alguna acción que podría provocar errores, desactivando los comandos que no son operativos en ese contexto.

Por último, se debe tener en cuenta, para atraer la atención del usuario y mejorar la interacción con el sistema, el uso de recursos estéticos como el color o el sonido. Éstos deben ser usados con cuidado, pero si se mezclan de forma coherente permiten conseguir muy buenos resultados.

2 Diseño de interfaces de aplicaciones.

2.1 Introducción

Actualmente, en nuestras tareas cotidianas interactuamos constantemente con interfaces de usuario, no solamente al usar los ordenadores personales sino también con otros objetos como el móvil, el cajero automático, la smart TV, etc. La interfaz con la que interactuamos tiene un papel fundamental para que el producto nos resulte fácil de manejar y, por ende, acabe teniendo éxito entre los usuarios y se consolide un producto competitivo en el mercado. Y es que el producto no tendrá buena aceptación si el usuario no consigue concretar una acción, no entiende la secuencia de pasos a seguir de manera lógica e intuitiva, no encuentra con facilidad cómo hacer la acción que necesita realizar, o no considera atractivo el diseño de la aplicación que está utilizando. A pesar de ello, el diseño de las interfaces de usuario no siempre ha sido considerada una tarea prioritaria en el desarrollo de aplicaciones software. Pero con el tiempo, al analizar porqué algunas aplicaciones fracasaban, se fue evidenciando que el diseño de la interfaz tiene una gran influencia en esto y, cada vez más, el diseño de interfaces de usuario está cobrando un mayor peso y relevancia entre las compañías que desarrollan productos software. Es más, en la actualidad, el diseño de interfaces de usuario es una parte fundamental en el proceso de desarrollo de cualquier aplicación, y, por lo tanto, tiene tanta importancia como el desarrollo o construcción de la aplicación en sí misma.

En este epígrafe se desarrolla en profundidad qué son las interfaces de usuario, los tipos que existen, los principios generales para realizar un buen diseño y su impacto en la usabilidad final del sistema. Asimismo, se verá qué son las guías de estilo y las herramientas y estándares relacionados.

2.2 Conceptos de interfaces de usuario

La interfaz de usuario es la parte del software que las personas pueden ver, oír, tocar, hablar; es decir, donde se pueden entender e interactuar con el sistema de información. Una interfaz de usuario tiene esencialmente dos componentes: la entrada y la salida.

- La entrada es cómo una persona le comunica sus necesidades o deseos al ordenador. Algunos componentes de entrada comunes son el teclado, el ratón, una pantalla táctil o un micrófono.
- La salida es la forma en que el ordenador transmite los resultados a lo solicitado por el usuario. Hoy en día el mecanismo de salida del ordenador más común es la pantalla, seguido de dispositivos que aprovechan las capacidades auditivas de una persona: de voz y sonido.

Como se verá más adelante, la Interfaz Gráfica de Usuario (en inglés, Graphic User Interface, ampliamente conocido por su acrónimo GUI), es un tipo de interfaz de usuario que facilita la interacción del usuario con el ordenador a través de la utilización de un conjunto de imágenes y objetos pictóricos (iconos, ventanas...), además de texto. Surge como evolución de la línea de comandos de los primeros sistemas operativos y es pieza fundamental en un entorno gráfico. Douglas Engelbart, además de inventor del ratón de ordenador, desarrolló la primera interfaz gráfica en los años 1960 en EE.UU. en los laboratorios de XEROX. Fue introducida posteriormente al público en los ordenadores Apple Macintosh en 1984, y a las masas hasta 1993 con la primera versión popular del sistema operativo Windows 3.0.

Existen tres puntos de vista distintos en una GUI:

- Modelo del usuario: el usuario tiene su propia visión del sistema y espera que se comporte de determinada manera. El modelo de usuario se puede conocer estudiándolo a través de test, entrevistas o realimentación.
- Modelo del diseñador: el diseñador es quien se encarga de unir las ideas, necesidades y deseos del usuario, con las herramientas que dispone el programador para desarrollar el software. Este modelo consta de tres partes: la *Presentación*, que es lo primero que llama la atención del usuario; la *Interacción*, que es donde el usuario constata si el producto satisface

sus expectativas; y las *Relaciones entre objetos*, donde se define la relación entre el modelo mental del usuario y los objetos de la Interfaz.

- **Modelo del programador:** es el modelo más fácil de visualizar porque se puede especificar formalmente. Este modelo consta de los objetos que manipula el programador, distintos a los que maneja el usuario, quien no ve los objetos que maneja el programador. Si bien el programador tiene conocimientos sobre las plataformas de desarrollo, sistemas operativos, lenguajes y herramientas de programación, especificaciones técnicas, ello no significa que tenga la habilidad necesaria para realizar el modelo correctamente desde el punto de vista del usuario.

De esta manera, los distintos modelos permiten conocer cómo visualizan la GUI cada uno de los 'actores', y cada uno de ellos son importantes para el diseño de la interfaz de usuario.

2.2.1 Tipos de interfaces de usuario

2.2.1.1 Según su construcción

Se pueden distinguir tres tipos de interfaces de usuario según los elementos que intervienen en su construcción:

- Una interfaz hardware, a nivel de los dispositivos utilizados para introducir, procesar y mostrar los datos: teclado, ratón y pantalla.
- Una interfaz software, destinada a mostrar información acerca de los procesos y herramientas de control, a través de lo que el usuario observa habitualmente en la pantalla.
- Una interfaz software-hardware, que establece un puente entre la máquina y las personas, permite a la máquina entender la instrucción y al hombre entender el código binario traducido a información legible.

2.2.1.2 Según la forma de interacción del usuario

Dentro del diseño de aplicaciones software, para diseñar una navegación eficiente de acuerdo al tipo de usuario, se puede elegir cualquiera de los siguientes tipos de interfaz.

Menús

Una interfaz de menú permite que el usuario elija las posibles opciones de una lista en pantalla. Al responder el usuario al menú, se limita solamente a las opciones que se le presentan. Para ello, el usuario no necesita conocer el sistema, pero sí necesita saber qué tareas pueden realizarse.

Los menús no son elementos dependientes del hardware, pero el dispositivo hardware utilizado para interactuar con las entradas de menú sí que determina la manera en que va ser usado: a través del teclado, de lápices ópticos o del ratón, y eventualmente por voz. Normalmente, las opciones del menú son identificadas mediante un número, una letra, una clave o atajo de teclado para facilitar su uso desde el teclado en lugar de usando el ratón.

Un aspecto importante cuando se diseñan menús en una aplicación es mantener la consistencia entre menús, y que acciones equivalentes se realicen siempre de la misma manera. Los menús pueden encontrarse anidados, o uno dentro del otro en cascada, de tal forma que el usuario los visualice conforme los va necesitando. Este tipo de menús permiten mantener la pantalla más sintetizada y despejada, facilitan un movimiento más rápido y eficiente por la pantalla y evitan que el usuario vea las opciones de menú que no sean de su interés.

En la siguiente figura puede verse un ejemplo de una barra de menú en la aplicación Microsoft Word:

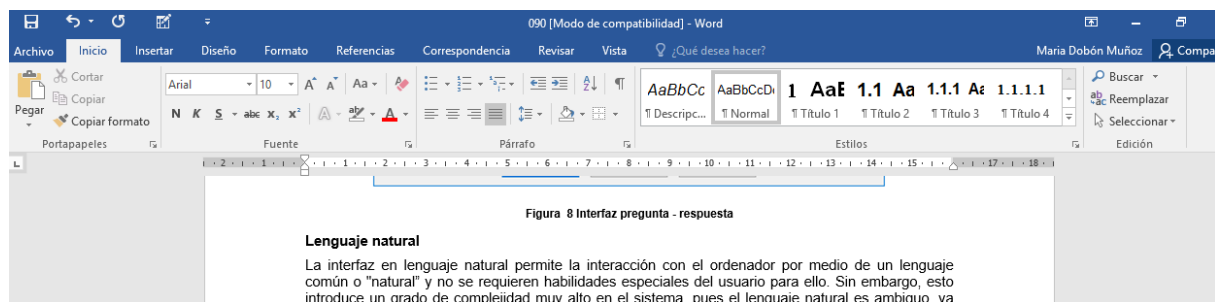


Figura 7 Ejemplo de menú

Interfaz de preguntas y respuestas

En este tipo de interfaces simplemente se muestran preguntas al usuario en pantalla para conocer qué acción desea realizar en ese momento. El usuario proporciona una respuesta (a través del teclado), el sistema realizará la acción correspondiente en función de la respuesta que haya dado el usuario y pasará a la siguiente pregunta si fuera necesario.

Cuando se diseñan las interfaces, se debe decidir el grado de flexibilidad con que contará el usuario en el momento de responder las preguntas, y el usuario debe conocer en todo momento el grado de flexibilidad que se le otorga. Por ejemplo, el usuario deberá saber si al teclear una "S" se sustituye de manera aceptable a la respuesta "Si". Por ello, es conveniente incluir ayuda adicional u orientación para recordarle al usuario qué respuestas son adecuadas en todo momento.

Un ejemplo muy común de interfaz de preguntas y respuestas es el usado en las ventanas de tipo emergente o pop-up:

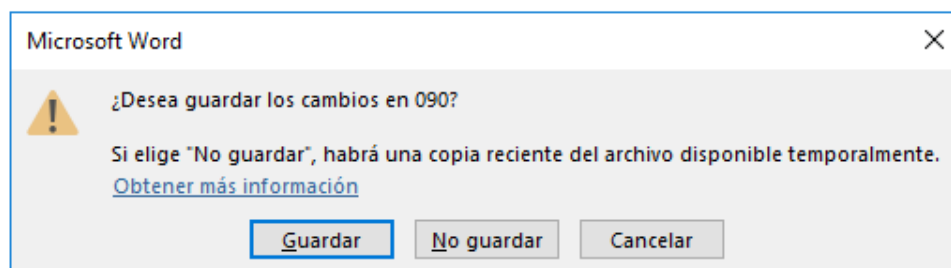


Figura 8 Interfaz pregunta - respuesta

Lenguaje natural

La interfaz en lenguaje natural permite la interacción con el ordenador por medio de un lenguaje común o "natural", y no se requieren habilidades especiales del usuario para ello. Sin embargo, esto introduce un grado de complejidad muy alto en el sistema, pues el lenguaje natural es ambiguo, pues está lleno de interpretaciones y de múltiples excepciones para cada una de las reglas.

De hecho, existen ciertos intentos limitados de interfaces en lenguaje natural para aplicaciones específicas donde otro tipo de interfaz sería menos operativa. Aunque los problemas de implicación y la demanda extraordinaria en recursos de cómputo han limitado considerablemente el desarrollo de las interfaces en lenguaje natural, numerosos programadores e investigadores están trabajando de manera activa en el desarrollo de interfaces en lenguaje natural, por lo que es un área que definitivamente continuará creciendo en el futuro. Hoy en día, ya se pueden ver asistentes como Siri en Apple, Cortana en Microsoft y Google Now en Android, que permiten interactuar con el sistema a través de lenguaje natural hablado.

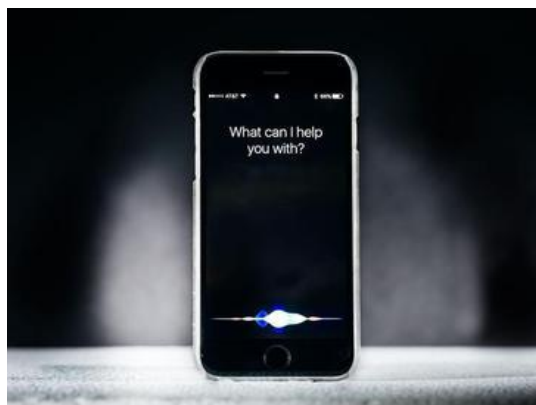


Figura 9 Imagen de teléfono con Siri

Formularios de entrada - salida

Los formularios de entrada/salida son formularios en pantalla que despliegan campos que contienen datos o parámetros que requieren ser comunicados o solicitados al usuario. El formulario, con frecuencia, es un 'facsimilar' de un formulario familiar para el usuario. Esta técnica también se conoce como interfaz basada en formularios.

Los formularios en pantalla muestran la información que deberá introducirse, así como su ubicación. Los campos en blanco que requieran de la entrada de información deben resaltarse del resto de los campos (mediante caracteres en video inverso o que destellan, o algún elemento visual distintivo). El usuario puede desplazar el cursor de campo en campo, mediante un sencillo juego de las teclas de flechas, mediante el cursor o la tecla de tabulador.

Los formularios por pantalla para la entrada de datos pueden ser simplificados proporcionando valores por omisión (default) en cada uno de los campos y permitir que el usuario modifique tales valores si así se considera necesario. Asimismo, se pueden introducir validaciones simples de cálculo o de formato para asegurar la correcta introducción de la información por el usuario, lo cual suele ser bastante útil para evitar almacenar datos incorrectos.

Este tipo de interfaces es muy habitual en el ámbito de las Administraciones Públicas, pues tradicionalmente se ha trabajado extensamente con formularios y solicitudes en papel, que son informatizadas y adaptadas al ámbito electrónico.

Figura 10 Ejemplo de formulario del Registro Electrónico Común

Lenguaje de comandos

La interfaz del lenguaje de comandos permite que el usuario maneje la aplicación mediante una serie de teclas, comandos, frases o cierta secuencia de los mismos. Se trata de una interfaz muy básica de manera visual pero muy popular entre los usuarios expertos, ya que normalmente permite realizar un abanico de acciones más amplio y dota a los usuarios de mayor flexibilidad y control que otras interfaces. No obstante, suele requerir de conocimientos medios/avanzados de informática y la memorización de reglas de sintaxis, lo cual puede ser un obstáculo para usuarios inexpertos (sin embargo, con la práctica cualquiera puede aprenderlos). Ejemplos de interfaces de este tipo son las consolas de comandos a través de las cuales se manejan los sistemas operativos Windows, Unix, Linux...etc.

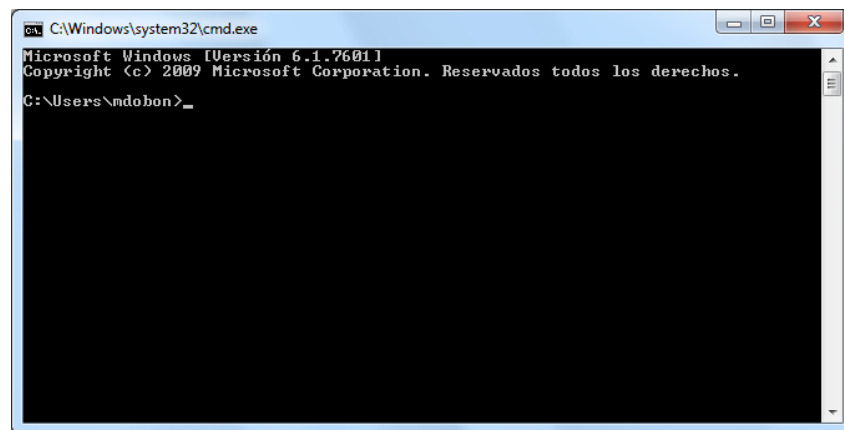


Figura 11 Ejemplo de interfaz de comandos CMD Windows

Interfaz gráfica

La interfaz gráfica está relacionada fundamentalmente con la tecnología utilizada y el soporte que brinde el sistema operativo o software de base utilizado (por ejemplo, Windows o KDE para Linux). La principal novedad que introduce este tipo de interfaces es el uso de iconos. Los iconos son símbolos o figuras que representan las acciones que el sistema puede realizar. Por ejemplo, la figura de una impresora representa la acción de impresión del sistema, la figura de un disquete significa guardar un archivo en disco, etc. Los iconos definen estándares que son utilizadas por todas las aplicaciones, y que son reconocidos fácilmente por los usuarios. La figura 9 mostrada anteriormente incluye varios ejemplos de iconos ampliamente utilizados en las aplicaciones de edición de textos.

Otro concepto relacionado con las interfaces gráficas es WIMP (Windows, Icons, Menus, Pointers), término acuñado por Merzouga Wilberts en 1980, que se refiere a un estilo de interacción de estos elementos de la interfaz de usuario. Aunque el término ha caído en desuso, algunas personas aún lo siguen utilizando como un sinónimo de interfaz gráfica. Cualquier interfaz que utiliza gráficos se puede denominar interfaz gráfica de usuario, y los sistemas de WIMP se derivan de este tipo de sistemas. Sin embargo, aunque todos los sistemas WIMP utilizan gráficos como un elemento clave (el icono y elementos de puntero), y por lo tanto son GUI, lo contrario no es cierto. Algunos GUIs no se basan en ventanas, iconos, menús y punteros. Por ejemplo, la mayoría de los teléfonos móviles representan acciones como iconos, y algunos pueden tener menús, pero muy pocos incluyen un puntero o permiten ejecutar programas en una ventana.

Interfaz de uso directo

Este tipo de interfaz se deriva de la interfaz gráfica. Permite la gestión directa de la representación gráfica en la pantalla como lo haría el usuario comúnmente con el objeto físico, el cual puede realizarse mediante el acceso por el teclado, ratón o joystick. El uso directo requiere de una mayor sofisticación del sistema, con respecto a las interfaces estudiadas con anterioridad y sirve de base para la realidad virtual.

Algunos ejemplos de interfaces de manipulación directa son: las calculadoras, la papelera de reciclaje, los ecualizadores musicales, juegos de video, etc.

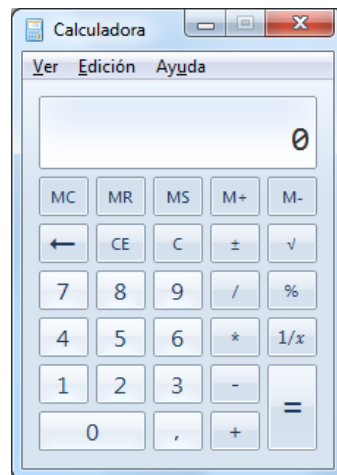


Figura 12 Ejemplo de interfaz de uso directo

Otras interfaces de usuario

Existen otras interfaces del usuario que, aunque son menos comunes que las descritas anteriormente, su popularidad no deja de crecer, especialmente con la generalización del uso de los smartphones. Dentro de estas interfaces tenemos los dispositivos de señalamiento, tales como los lápices ópticos, las pantallas táctiles y los sistemas de reconocimiento de voz y síntesis del lenguaje. Cada una de estas interfaces cuenta con atributos especiales que las hacen únicas para aplicaciones particulares.

2.3 Diseño de interfaces de usuario

El diseño de las interfaces de aplicaciones, o interfaces de usuario como comúnmente es conocida esta disciplina en la ingeniería del software, consiste en el diseño de las aplicaciones, máquinas, dispositivos, móviles, aplicaciones software, ordenadores y sitios web con interacción hombre-máquina. Se trata de una actividad multidisciplinar, ya que involucra diferentes sectores como el diseño gráfico, industrial, ergonomía, software...etc.

El principal objetivo del diseño de interfaces de usuario es que las aplicaciones sean más atractivas para el uso, así como facilitar la interacción con el usuario, haciéndola lo más intuitiva posible, o lo que es comúnmente conocido como **diseño centrado en el usuario**, que ha sido desarrollado en el tema correspondiente de accesibilidad y usabilidad.

Dos de las leyes más conocidas en el diseño de interfaces de usuario son la **Ley de Hick** y la **Ley de Fitts**. La ley de Hick explica que el tiempo de decisión de un humano varía según el número de opciones que se le presenten. Su formulación redactada sería la siguiente:

“El tiempo que se tarda en tomar una decisión aumenta a medida que se incrementa el número de alternativas.”

La Ley de Fitts, por su parte, trata del tiempo que tarda un humano en apuntar a un determinado punto. La formulación de la ley de Fitts sería:

“El tiempo que se requiere para alcanzar a pulsar un objetivo depende de una relación logarítmica entre su superficie y la distancia a la que se encuentra.”

Ambas leyes se complementan, ya que el usuario primero elegiría una opción de las disponibles y luego llegaría a ella a través de la interfaz.

Asimismo, tal y como se mencionó anteriormente, el usuario es un actor fundamental en el momento de diseñar la interfaz, por lo tanto, antes de tomar decisiones respecto al diseño, se deben tener en cuenta las habilidades físicas y mentales del mismo. Existen numerosos **factores humanos** que no se deben ignorar, como que las personas tienen una memoria limitada a corto plazo, la facilidad para

cometer errores especialmente cuando se trabaja de manera rápida o bajo presión, las diferentes capacidades físicas que pueden tener las personas e incluso las diferentes preferencias personales de los usuarios en la interacción hombre-máquina.

Todas estas leyes y factores, entre otras consideraciones, deben tenerse en cuenta en el diseño de una interfaz de usuario y forman parte de los principios generales del diseño de una interfaz gráfica de usuario (GUI) que se exponen a continuación.

2.3.1 Principios de diseño de interfaces de usuario

Aunque existe una gran cantidad de información y bibliografía respecto al diseño de la GUI, en este apartado se explican de manera resumida los principios más relevantes:

- Familiaridad del usuario: significa que la interfaz debe utilizar términos e imágenes conocidos por el usuario; y los objetos que manipula el sistema deben estar relacionados con el ámbito de trabajo.
- Uniformidad de la interfaz: significa que tanto comandos como menús deben tener el mismo formato. Las interfaces uniformes reducen el tiempo de aprendizaje.
- Mínima sorpresa: el comportamiento del sistema no debe mostrar situaciones inesperadas. Ante este tipo de situaciones el usuario puede mostrar irritabilidad, y, por lo tanto, perder interés en utilizar la aplicación.
- Recuperación de estados: este es uno de los principios más importantes al diseñar una interfaz. Es inevitable cometer errores, por lo tanto, el sistema le debe proporcionar al usuario la manera de subsanarlos o volver a estados anteriores. Este principio involucra varias acciones como pedir al usuario que confirme acciones destructivas, que el usuario pueda deshacer, etc.
- Guía de usuarios: la interfaz debe proporcionar al usuario asistencia, ayuda. No solamente cuando se cometen errores sino también cuando no se sabe qué hacer o cómo hacer alguna tarea. Esta ayuda debe estar integrada al sistema y debe ser clara cuando el usuario la requiera, sin saturar con información.
- Diversidad de usuarios: se deben tener en cuenta los diferentes usuarios que pueden utilizar la aplicación. Aquellos casuales, que necesitan que los guíen, y aquellos que podrían usarla constantemente los cuales necesitarán trabajar con métodos abreviados, tan rápido como sea posible. Además, se podría incluir recursos para mostrar diferentes tamaños de texto, reemplazar sonido por texto y al revés, modificar tamaño de botones, etc. Esto refleja la noción de Diseño Universal, principio de diseño cuyo objetivo es evitar excluir usuarios.
- Adoptar el punto de vista del usuario: se debe ver la interfaz desde fuera y en relación con las tareas que va a realizar el usuario. Hay que tener mucho cuidado en no centrarse en los aspectos de implementación que hagan perder la perspectiva.
- Realimentación: la interfaz debe dar inmediatamente alguna respuesta a cualquier acción del usuario. Por ejemplo: movimiento del cursor, resaltar la opción elegida de un menú, comunicar el éxito o fracaso de una operación, reflejar el estado de los objetos.
- Potenciar la sensación de control del usuario sobre el sistema, especialmente para los usuarios sin experiencia: que la interfaz sea intuitiva (utilizar iconos, modelos, métodos, etc. consistentes con otras aplicaciones y con el mundo real), facilitar la exploración (todas las operaciones deben ser accesibles desde el menú principal), permitir cancelar y deshacer operaciones, etc.
- Minimizar la necesidad de memorización: usar controles gráficos, limitar la carga de información a corto plazo, procurar que la información necesaria en cada momento esté presente en la pantalla, utilizar nombres y símbolos auto-explicativos y fáciles de recordar, etc.
- Anticipación: la aplicación debe anticiparse a las necesidades del usuario, y no esperar a que tenga que buscar información.

- Percepción de color y tamaño: se debe tener en cuenta a aquellos usuarios con problema de visualización del color, pero es muy útil usar convención de colores. Además, al mostrar varios objetos en la pantalla deben estar distribuidos, debe haber distancia entre ellos para que así el usuario pueda percibirlos sin sobrecarga.
- Legibilidad: no solamente se debe prestar atención a los colores y a los objetos que se ven en pantalla sino también a cómo se verá el texto. El tipo y tamaño de letra debe ser legible, y el color debe contrastar con el fondo (utilizar letras negras en fondo claro).
- Valores por defecto: lo ideal es utilizar 'valores estándar'. Se debe tener en cuenta que los valores por defecto deben ser opciones inteligentes, sensatas y fáciles de modificar.
- Eficiencia: se debe considerar la productividad como ideal a lograr. El usuario no debe esperar la respuesta del sistema por tiempo prolongado; los mensajes de ayuda, menús y etiquetas deben ser sencillos y deben utilizar palabras claves para poder transmitir fácilmente a qué hacen referencia.

2.3.2 Enfoques de diseño UI, UX, IxD

Conforme el diseño de interfaces usuario y su impacto en la usabilidad ha ido extendiéndose y evolucionando, han ido surgiendo nuevos enfoques o variantes que han incorporado elementos y visiones a tener en cuenta en el diseño de interfaces, con el objetivo último de mejorar la experiencia de usuario. Así es como han surgido las siguientes disciplinas de diseño de interfaces UI, UX y IxD, donde cada una de ellas pone su foco en un elemento diferente.

Diseño de interfaz de usuario (UI User Interface)

Este enfoque considera que todos los elementos que te permiten interactuar con un dispositivo forman parte del diseño del User Interface, la interfaz de usuario. La interfaz de usuario es uno de los elementos que componen la experiencia de usuario y ayuda principalmente a navegar por la página o aplicación fácilmente. Se preocupa especialmente de la selección, distribución y apariencia de los elementos con los que interactúan los usuarios en una plataforma. Su finalidad es guiar a los usuarios para que realicen las acciones que se definieron como objetivos de un proyecto.

Entre sus tareas están la definición de la identidad visual, la correcta adaptación a los distintos soportes y la comunicación de jerarquías de contenidos y opciones en una plataforma. Además, debe integrar las estrategias SEO (Search Engine Optimization), cumplir con estándares (HTML, CSS) y asegurar la accesibilidad de todos los usuarios, independientemente de sus características.

Los principios fundamentales en el diseño de UI son:

- Los elementos del diseño deben ser totalmente claros para el usuario.
- El diseño debe ser flexible, así será adaptable a diversas situaciones.
- Diseño simple y familiar. Utiliza elementos convencionales que sean fáciles para el usuario.
- El diseño debe ser eficiente para que el usuario complete las tareas correcta y rápidamente.
- Elementos estructurados y jerarquía visual.

Diseño de experiencia de usuario (UX User eXperience)

Como nos indica su propio nombre, UX es la experiencia de una persona, un usuario, cuando interactúa con la página web o aplicación. La clave para llevar a cabo estrategias de UX es situar al usuario en el centro del diseño, centrándose en sus necesidades, habilidades y limitaciones. Todos los elementos visuales, como botones e iconos, contenido, funciones, respuestas, que nos ayudan a realizar las acciones que deseamos forman parte de la UX. Su foco es responder de la mejor manera posible a las necesidades y comportamientos de las personas. Su finalidad es ayudarlos a realizar las acciones.

UX es un concepto muy amplio, dentro de la UX encontramos varios elementos importantes que desarrollar como la arquitectura de información, el diseño de la interacción y el diseño visual del sistema.

La arquitectura de la información (IA) es la disciplina responsable de crear estructuras de contenidos. Para ello, debe considerar el comportamiento de los usuarios al buscar y acceder a la información, para responder de forma acertada a sus consultas y facilitar las tareas. Para lograr esto, estudia, analiza, organiza y jerarquiza los datos a través de ejercicios de *cardsorting*, mapas de navegación y *wireframes*.

Los principios fundamentales del diseño de la Experiencia de Usuario son:

- El contenido de la web debe ser útil para el usuario.
- La página web debe ser usable.
- Debemos transmitir una imagen deseable (identidad, marca, emociones).
- Contenido “encontrable”, fácil de navegar.
- Contenido accesible para personas con discapacidad.
- Contenido creíble, el usuario debe confiar en ti.

Diseño de interacción (IxD)

El diseño de interacción es la disciplina que busca definir la forma en que se utilizan los productos y servicios interactivos, define cómo responderá una interfaz a las acciones con los usuarios, estableciendo flujos de operación y puntos de contacto. El objetivo del diseño de interacción es intentar que las personas consigan realizar las acciones o tareas para las que el producto ha sido diseñado de la manera más sencilla, intuitiva y gratificante posible. Para esto, debe considerar todos los aspectos necesarios según las particularidades del usuario, desde su edad y características físicas hasta necesidades, deseos o aspectos sociales, culturales o el contexto en el que se piensa que el producto puede ser utilizado.

Para lograr esto, esta disciplina crea partituras de interacción y realiza pruebas con usuarios finales. Es durante estos test cuando el diseñador podrá observar la manera en que la persona se enfrenta al producto o dispositivo, cómo interactúa con él y si consigue cumplir las tareas que se propone. Esta instancia de observación permitirá al equipo de diseño obtener respuestas por parte de las personas que usarán sus productos y modificar los errores que puedan ser identificados. La usabilidad, otro concepto relacionado con el tema, apoyada en aspectos propios de la percepción y la cognición, entre otras, utiliza métodos fiables de evaluación de los objetos interactivos que buscan medir el nivel de comprensión, satisfacción y facilidad con que las personas usan esos objetos.

Los elementos principales del diseño de interacción son:

- Contenido escrito. Nuestras palabras tienen gran influencia en el usuario y son cruciales para guiar la interacción.
- Elementos visuales. No debemos saturar de texto, los elementos visuales aportan sencillez.
- El objeto de interacción. Según el dispositivo hay unas limitaciones de espacio (no es lo mismo la pantalla de un móvil que la de un ordenador).
- Comportamiento del usuario. Ten en cuenta las emociones y reacciones del usuario mientras interactúa con el sistema.

2.4 Usabilidad

El diseño de interfaces de aplicaciones está estrechamente relacionado con la usabilidad del software. Según ISO/IEC 9241, la usabilidad es la eficacia, eficiencia y satisfacción con la que un producto. Otra definición del término es que se trata de una medida de la calidad de la experiencia que tiene un usuario cuando interactúa con un producto o sistema. Esto es medido a través del estudio de la relación que se produce entre las herramientas (entendidas en un sitio web como el conjunto integrado por el sistema de navegación, las funcionalidades y los contenidos ofrecidos) y quiénes las utilizan, para determinar la eficiencia en el uso de los diferentes elementos ofrecidos en las pantallas y la efectividad en el cumplimiento de las tareas que se pueden llevar a cabo a través de ellas.

La usabilidad idealmente debe considerarse desde el inicio del proceso de desarrollo hasta las últimas acciones antes de implementar el sistema, producto o servicio disponible al público. Antes de iniciar el proyecto es esencial tener una idea acerca de las características de los usuarios y de los aspectos del producto en cuanto al interés y necesidad que podría tener el usuario. Teniendo en cuenta estas consideraciones de usabilidad de forma temprana se ahorra tiempo y dinero, dado que la posterior implementación de nuevos aspectos o nuevas interfaces de usuario implican un enorme esfuerzo adicional si ya está desarrollado. Es precisamente durante el proceso de diseño de las interfaces de usuario donde se debe poner especial énfasis en los principios de la usabilidad, y además, durante todo el desarrollo es conveniente realizar pruebas para comprobar que se está considerando adecuadamente la usabilidad del producto, incluso una vez que el producto está en el mercado se debería consultar a los usuarios acerca de sus necesidades y actitud respecto del mismo para intentar recoger feedback y realizar las mejoras pertinentes si así se considera necesario.

La usabilidad del software ha sido desarrollada en el tema dedicado a la accesibilidad y usabilidad web, por lo que para más información sobre el concepto, principios y estándares de usabilidad se recomienda revisar dicho tema.

2.5 Herramientas y lenguajes para el desarrollo de interfaces de usuario

Cualquier lenguaje de programación moderno ofrece herramientas para la construcción de interfaces gráficas de usuario (GUI). A continuación, se listan algunos de los lenguajes y herramientas más relevantes.

GLADE

Es una herramienta de desarrollo visual de interfaces. Permite construir de forma gráfica e interactiva interfaces de usuario gráficos para Gnome/Gtk, publicado bajo la licencia GNU GPL. Es independiente del lenguaje de programación y no genera código fuente, sino un archivo XML que representa la interfaz de usuario y que posteriormente puede ser utilizado para construirla en tiempo de ejecución.

GTK+ (GIMP Toolkit)

Es una biblioteca para crear interfaces graficas de usuario. Su licencia es la LGPL. GTK está construido encima de GDK (GIMP Drawing Kit), que básicamente es un recubrimiento de las funciones de bajo nivel que debe haber para acceder al sistema de ventanas sobre el que se programe. Fue escrito para el desarrollo del General Image Manipulation Program (GIMP), pero ahora GTK se utiliza en un gran número de proyectos de programación, incluyendo el proyecto GNU Network Object Model Environment (GNOME).

GTK es esencialmente una interfaz para la programación de aplicaciones orientadas a objetos (API). Aunque está completamente escrito en C, se ha implementado haciendo uso del concepto de clases y de funciones respuesta o de callback (punteros o funciones).

TCL/TK (Tool Command Language)

Es un lenguaje de programación interpretado y multiplataforma. Es distribuido de forma totalmente gratuita, aunque su uso sea para aplicaciones comerciales, a través de Internet.

Una de sus principales características es su gran facilidad con la que se pueden implementar funciones en C/C++ que pasan a ser nuevas instrucciones del intérprete. La extensión más conocida, y que es distribuida junto con el propio TCL, es TK (Tool Kit). TK añade a los comandos de TCL, comandos capaces de crear interfaces gráficos de usuario.

TCL se utiliza principalmente para el desarrollo rápido de prototipos, aplicaciones script, interfaces gráficas y pruebas, y la combinación con TK se utiliza para crear interfaces gráficas.

WXWidgets

Son unas bibliotecas libres y multiplataforma para el desarrollo de interfaces gráficas programadas en C++. Proporcionan una interfaz gráfica basada en las bibliotecas ya existentes en el sistema (nativas). Están disponibles para Windows, MacOS, GTK+, Motif, OpenVMS y OS/2. También pueden ser utilizadas desde otros lenguajes de programación: Java, Javascript, Perl, Python...etc.

Se distribuye bajo licencia wxWindows Library License, que es similar a la GNU Library General Public License pero que, además, permite usar la biblioteca para desarrollos comerciales (ya sean aplicaciones o modificaciones de la propia biblioteca), siempre y cuando estos desarrollos comerciales no usen ningún código distribuido bajo alguna licencia GNU.

JFC (Java Foundation Classes) de Java

Son parte de la API de Java, compuestas por clases que sirven para crear interfaces gráficas visuales para las aplicaciones y applets de Java. Las JFC contienen varios paquetes gráficos, como Swing para los componentes de los botones y menús, y Java 2D para dibujar gráficos en dos dimensiones. No obstante, ya se han desarrollado nuevos paquetes gráficos en el entorno Java, como SWT y Java FX.

SWT (Standard Widget Toolkit)

Es un conjunto de componentes para construir interfaces gráficas en Java, desarrollados por el proyecto Eclipse, manteniendo la idea de ya obsoleto AWT (Abstract Window Toolkit) de utilizar componentes nativos de la plataforma.

Java FX

Es una familia de productos de Oracle para la creación de aplicaciones RIA (Rich Internet Applications), es decir, aplicaciones web que tienen las características de aplicaciones de escritorio. Pueden ser ejecutadas en una amplia variedad de dispositivos.

2.6 Guías de estilo

Para poder asegurar consistencia a través de las diferentes partes de un sistema o a través de una familia de sistemas, es fundamental para los desarrolladores basar sus diseños en un conjunto de principios y directrices. Esto permite transferir sus conocimientos a los usuarios de la interfaz, dentro de un producto y a todas las aplicaciones en que trabaja. Por este motivo es tan importante para las organizaciones que desarrollan software disponer de una guía que puedan seguir sus desarrolladores. Estas guías se denominan guías de estilo y varían mucho en sus objetivos. Además, hay que tener en cuenta que los estilos pueden ser diferentes en función de la plataforma en la que se despliega la aplicación: web, app, escritorio o móvil, y normalmente tendrán diferencias de estilo adecuadas a sus particularidades.

La ventaja más evidente de las guías de estilo es que aseguran una mejor usabilidad gracias a la consistencia que imponen. En el lenguaje industrial al hablar de un cierto tipo de guía de estilo se le suele dar también la denominación de *look and feel*.

Las hay de dos tipos: guías de estilo comerciales y guías de estilo corporativas.

2.6.1 Comerciales

Son guías de estilo diseñadas por las empresas de software. Están basadas en principios y contienen directrices que se concretan a muy bajo nivel. Las guías de estilo más relevantes, que están basadas en los entornos operativos más importantes, son: Macintosh, OS/2, Windows y UNIX. Estos sistemas son propiedad de las empresas informáticas y cada una de ellas publica su guía de estilo propia, que va modificando y adecuando a los nuevos sistemas operativos o plataformas que va liberando.

Por ejemplo, Apple publicó su primera guía en el año 1985, *Human Interface Guidelines: The Apple Desktop Interface en Inside Macintosh*, con más de 600 páginas. Hoy en día, dispone de las *Human Interface Guidelines*, que contemplan información detallada sobre los recursos de UI para diseñar apps móviles o aplicaciones para cualquiera de sus plataformas.

Microsoft, por su parte, publicó en 1992, su primera guía en solitario: *Microsoft. The Windows Interface: An Application Design Guide*, que ha ido adecuando a las diferentes versiones del sistema operativo Windows hasta la actualidad.

En cuanto a Java, el *look and feel* representa la interfaz por defecto para las aplicaciones construidas con sus bibliotecas, y se ha publicado una guía de estilo *Java Look and Feel Design Guidelines* que proporciona información esencial para construir interfaces gráficas de usuario para aplicaciones con el lenguaje de programación Java y JFC. La primera versión de estas guías fue publicada en julio de 1999.

Otras guías conocidas son *GNOME User Interface Guidelines* o *KDE User Interface Guidelines*.

2.6.2 Corporativas

Las guías de estilo corporativas se centran en presentaciones comunes, comportamientos y técnicas que deben ser implementadas por todos los productos en una compañía. Un objetivo de las guías de estilo corporativas es mantener y reforzar la identidad corporativa, esto es, el uso de colores, gráficos, e iconos que presenten una imagen visual consistente del logotipo de la compañía a través de todas las interfaces de los productos. El método sugerido para la construcción de una guía de estilo corporativa consiste en basarse en una guía de estilo de la industria.

En la Administración General del Estado, existe la *Guía de Comunicación Digital*, que contempla directrices y guías de diseño a alto nivel para las páginas web dentro de su ámbito de competencia.

2.7 Estándares relacionados

A continuación, se listan los estándares relacionados más relevantes:

ISO 13407 – Procesos de Diseño centrados en la persona para sistemas interactivos.

Este estándar proporciona una orientación sobre las actividades de diseño centradas en la persona a lo largo del ciclo de vida de sistemas interactivos basados en ordenadores. Describe el diseño centrado en el usuario como una actividad multidisciplinar que incorpora factores humanos, y técnicas y conocimientos ergonómicos, con el fin de conseguir efectividad y eficiencia, y mejorar las condiciones de trabajo para las personas.

ISO/IEC 14598 – Evaluación de Productos de Software.

Este estándar comprende seis partes que especifican el proceso a seguir para evaluar software. La primera parte incluye la definición original de la calidad en el uso.

ISO 9241 – Requisitos ergonómicos para trabajar con terminales de presentación visual.

El estándar ISO 9241 (Ergonomic Requirements for Office Work with Visual Display Terminal) provee una guía en usabilidad, introduciendo requerimientos y recomendaciones para ser utilizadas durante el diseño de una aplicación y durante su evaluación. Está compuesto de 17 partes que tratan cuatro áreas principales:

- Introducción y descripción general (partes 1–2)
- Requisitos hardware (partes 3–9)
- Requisitos del entorno (parte 6)

- Requisitos software (partes 10–17)

ISO/IEC 9126 - Evaluación de productos software: características de calidad y directrices para su uso.

La norma ISO 9126, ahora bajo el proyecto SQuaRE (Software Product Quality Requirements and Evaluation), en que desarrolla la norma **ISO 25000**, establece un modelo de calidad basado en modelos de calidad propuestos por los investigadores durante los últimos 30 años.

La ISO 9126, y su sucesora, la ISO 25000, propone un modelo de calidad que se divide en tres aproximaciones: interna (calidad del código), externa (calidad en la ejecución) y en uso. En este caso, el modelo establece diez características, seis de ellas comunes a las vistas interna y externa y las otras cuatro propias de la vista en uso.

ISO 13406 - Requisitos ergonómicos para trabajar con presentaciones visuales basadas en paneles planos.

También especifica métodos para determinar la calidad de la imagen.

ISO/IEC 10741 - Interacción de diálogos.

Define cómo deben iniciar, controlar y monitorizar los usuarios las posibilidades del sistema para controlar los cursores en sistemas de texto por medio de funciones de control.

ISO/IEC 11581 - Símbolos y funciones de los iconos.

Comprende 6 partes que se aplican a los iconos que son visualizados en la pantalla del ordenador. Estos iconos representan datos o funciones del sistema con los que los usuarios pueden interactuar o manipular.

ISO 11064 - Diseño ergonómico de los centros de control.

Está dividido en 8 partes que contienen los principios ergonómicos, recomendaciones y directrices para el diseño de los centros de control.

W3C Web Accessibility Initiative.

W3C es un consorcio internacional que promueve la evolución e interoperabilidad en la Web. Uno de sus dominios de actuación es la accesibilidad, y por ello alberga la Iniciativa de Accesibilidad Web (Web Accessibility Initiative, WAI), que está patrocinada por una gran variedad de organizaciones preocupadas por la accesibilidad. Las guías juegan un papel muy importante de cara a conseguir una Web accesible explicando cómo emplear tecnologías Web para crear sitios Web accesibles, browsers o herramientas autorizadas. WAI tiene tres guías diferentes para solucionar estas necesidades: Web Content Accessibility Guidelines, Authoring Tool Accessibility Guidelines y User Agent Accessibility Guidelines. Estas guías se detallan ampliamente el tema de accesibilidad.

3 Conclusión

Como se ha visto, las interfaces hombre – máquina ya se han configurado como una de las áreas de investigación crítica para el desarrollo de la sociedad de la información. La usabilidad es reconocida actualmente como un importante atributo de la calidad del software, ganando éste lugar entre atributos más tradicionales tales como el rendimiento y la robustez. De hecho, diversos programas académicos llevan ya tiempo enfocándose en las interfaces gráficas de usuario y la usabilidad. Además, han surgido muchas compañías consultoras de usabilidad, y las consultoras tradicionales, así como las firmas de diseño, están ofreciendo servicios similares.

El diseño de interfaces involucra un proceso iterativo el cual significa que no se deben dejar las pruebas hasta tener un diseño final, ya que sería imposible arreglar todos los problemas. Es mejor realizar diseño de prototipos en tiempo de desarrollo, ejecutar pequeñas pruebas y revisar el diseño de cada una, de forma que se reparen los defectos identificados cuanto antes.

Así, el diseño de interfaces de usuario usables es una condición necesaria para sobrevivir en el caso de los sitios web, y especialmente los relacionados con la administración electrónica, ya que, si el sitio no logra establecer claramente lo que el usuario puede hacer, presenta información difícil de leer o no contesta las dudas del usuario, el ciudadano no logrará realizar el trámite deseado. En estas circunstancias, la primera línea de defensa del mismo será abandonar el sitio y no utilizarlo, optando por seguir relacionándose con las Administraciones Públicas de la manera tradicional, la ya conocida, que, aunque más costosa en tiempos de desplazamiento y plazos de tramitación, les ofrece la seguridad y confianza de la que un sistema con una interfaz poco intuitiva muchas veces adolece.

4 Glosario

GUI: Graphical User Interface

IGU: Interfaz Gráfica de Usuario

GUI: Graphical User Interface

WIMP: Windows, Icons, Menus, Pointer

UX: User Experience

JFC: Java Foundation Classes

AWT: Abstract Window Toolkit

GTK Gimp Toolkit

UI User Interface

IXD Interaction Design

WAI Web Accesibility Initiative

GNOME GNU Network Object Model Environment

HTML HyperText Markup Language

CSS Cascading Style Sheets

KDE K Desktop Environment

SEO Search Engine Optimization

W3C World Wide Web Consortium

ISO International Organization for Standardization

TCL/TK Tool Command Language TookKit

RESUMEN ESQUEMÁTICO

- LA ELABORACIÓN DE PROTOTIPOS EN EL DESARROLLO DE SISTEMAS
 - Introducción: el prototipado es una técnica de análisis de requisitos que se ha desarrollado para facilitar el entendimiento entre usuarios y desarrolladores.
 - Conceptos de prototipos:
 - Definición: producto software que se obtiene en una fase anterior a la de finalización de un proyecto de desarrollo, con objeto de permitir evaluar, modificar las especificaciones y demostrar al usuario en qué punto del desarrollo se encuentra.
 - Objetivo: un prototipo permite demostrar conceptos, probar opciones de diseño e informarse más sobre el problema y sus posibles soluciones.
 - Beneficios: mejora de la usabilidad del sistema, mejor concordancia entre el sistema y las necesidades del usuario, mejora de la calidad del diseño, mejora en el mantenimiento y reducción del esfuerzo de desarrollo.
 - Coste: un prototipo no implica un mayor coste del proyecto, pues se reduce el tiempo dedicado a reconstrucción y cambios en funcionalidades ya implementadas debido a cambios tardíos en los requisitos. No en todos los proyectos es adecuado utilizar prototipado.
 - Tipos de prototipos:
 - Según la función: de viabilidad, de necesidad, de diseño y de implantación.
 - Según la funcionalidad implementada: sketch, mockup, wireframe, maqueta, prototipo.
 - Según el destino: desechables y evolutivos.
 - Cuándo usar prototipado: en proyectos pequeños y medianos, proyectos con poca duración, cuando los requisitos no están claros, para funcionalidad innovadora, tecnologías nuevas.
 - Ventajas de los prototipos: mejora comunicación usuario-desarrollador, revisión continua de requisitos, usuarios son partícipes del desarrollo, mejora la formación.
 - Desventajas de los prototipos: se descartan alternativas desde el comienzo, menos flexibles a cambios posteriores, confusión del cliente, visión parcial del sistema por el usuario.
 - Modelos de ciclo de vida relacionados: modelo iterativo de prototipado y modelo en espiral.
 - Herramientas de prototipado:
 - Técnicas de cuarta generación
 - Componentes software reutilizables
 - Especificaciones formales y entornos para prototipos
 - Prototipado en Métrica v3: se encuentra integrado en varias fases, actividades y tareas. Métrica v3 contempla un modelo de gestión de proyectos por prototipos y lo usa como técnica en las fases de análisis ASI y diseño DSI.

- DISEÑO DE INTERFACES DE APLICACIONES

- Introducción: la interfaz de usuario tiene un papel fundamental para que el producto nos resulte fácil de manejar y, por ende, acabe teniendo éxito entre los usuarios.
- Conceptos de interfaces de usuario
 - Interfaz es la parte del software que las personas pueden ver, oír, tocar, hablar, donde pueden interactuar con el sistema de información.
 - Se compone de la entrada y la salida.
 - Existen tres puntos de vista: del usuario, del diseñador y del programador.
- Tipos de interfaces de usuario
 - Según su construcción: hardware, software, hardware-software.
 - Según la forma de interacción del usuario: menús, interfaz preguntas-respuestas, lenguaje natural, formularios de entrada-salida, lenguaje de comandos, interfaz gráfica, de uso directo.
- Diseño de interfaces de usuario
 - Ley de Hick: el tiempo que se tarda en tomar una decisión aumenta a medida que se incrementa el número de alternativas.
 - Ley de Fitts: el tiempo que se requiere para alcanzar a pulsar un objetivo depende de una relación logarítmica entre su superficie y la distancia a la que se encuentra.
 - Principios: familiaridad del usuario, uniformidad de la interfaz, recuperación de estados, guía de usuarios, diversidad de usuarios, adoptar el punto de vista del usuario, realimentación, potenciar la sensación de control del usuario sobre el sistema, minimizar memorización, anticipación, percepción de color y tamaño, legibilidad, valores por defecto, eficiencia.
 - Enfoques de diseño
 - UI: considera que todos los elementos que te permiten interactuar con el dispositivo forma parte de la interfaz.
 - UX: sitúa al usuario en el centro del diseño.
 - IxD: se centra en el diseño de la interacción hombre-máquina.
- Usabilidad: concepto estrechamente relacionado con el diseño de interfaces.
- Herramientas y lenguajes para el desarrollo de interfaces de usuario: GLADE, GTK+, TCL/TK, WXWidgets, JCF, SWT, JavaFX.
- Guías de estilo
 - Comerciales: de empresas software como Apple, Microsoft, Java.
 - Corporativas: propias de cada organización. Guía de Comunicación de la Administración General del Estado.
- Estándares relacionados: ISO 13407, ISO 14598, ISO 9241, ISO 9126-ISO 25000, ISO 13406, ISO 10741, ISO 11581, ISO 11064, W3C WAI.
- Conclusión: El diseño de interfaces de usuario usables es una condición necesaria para sobrevivir en el caso de los sitios web, y especialmente los relacionados con la administración electrónica, ya que, si el sitio no logra establecer claramente lo que el usuario puede hacer, presenta información difícil de leer o no contesta las dudas del usuario, el ciudadano no logrará realizar el trámite deseado

PREGUNTAS DE TEST

1) WIMP es:

- a) Un sinónimo de Interfaz Gráfica de Usuario.
- b) Windows, Icons, Menus, Pointer
- c) Una evolución de Diseño User Experience (UX)
- d) Un modelo de prototipado rápido.

2) Un Mockup es:

- a) Un tipo de prototipo.
- b) Una herramienta para evaluar la usabilidad de una interfaz.
- c) Una herramienta para realizar maquetas online.
- d) Un error típico en el diseño de interfaces de usuario.

3) En Métrica v3, la Definición de Interfaces de Usuario es la actividad:

- a) DSI 4
- b) PSI 5
- c) ASI 8
- d) CSI 3

4) La Ley de Hicks dice:

- a) El tiempo que se requiere para alcanzar a pulsar un objetivo depende de una relación logarítmica entre su superficie y la distancia a la que se encuentra.
- b) El tiempo que se tarda en tomar una decisión aumenta a medida que se incrementa el número de alternativas.
- c) El número de pasos para llegar a una función en una interfaz no debe exceder de 6.
- d) El tiempo invertido en el aprendizaje de uso de una aplicación es inversamente proporcional al dedicado al diseño de su interfaz.

5) Los prototipos según su función pueden ser:

- a) De viabilidad, de necesidad, de implantación y de diseño.
- b) Desechables y evolutivos.
- c) Maquetas, prototipos, wireframes, mockups.
- d) De diseño, de análisis, de construcción.

6) Respecto a la técnica del prototipado, señale la respuesta correcta:

- a) Puede desecharse después de su uso y comenzar el desarrollo del sistema.
- b) Es el sistema en sus fases iniciales, posteriormente debe evolucionar y ser completado con la funcionalidad requerida.
- c) El desarrollo resulta siempre más costoso.

d) No se puede utilizar en el modelo de ciclo de vida en cascada.

7) ¿Cuál de los siguientes es un estándar de usabilidad?

- a) ISO 12207
- b) UNE 139803
- c) ISO 9241
- d) ISO 9001

8) Una de las ventajas principales del uso del prototipado es que:

- a) Los tiempos de desarrollo se reducen.
- b) Los costes del desarrollo se reducen.
- c) Mejora la comunicación usuario-desarrollador.
- d) Se cumple mejor la planificación del sistema.

9) El diseño UX:

- a) Es una disciplina de diseño de interfaces de usuario cuyo fin último es la satisfacción del usuario.
- b) Es una metodología ágil de diseño de interfaces de usuario.
- c) Pone su enfoque en el diseño de la interacción hombre-máquina.
- d) Su pilar fundamental es la elaboración de prototipos.

10) Señale cuál de las siguientes es una herramienta de prototipado:

- a) Prototype pro.
- b) GTK
- c) Invision.
- d) Ease.ly

11) La disciplina que busca definir la forma en que se utilizan los productos y servicios interactivos, define cómo responderá una interfaz a las acciones con los usuarios, estableciendo flujos de operación y puntos de contacto, es:

- a) User Interface
- b) User eXperience
- c) Interaction Design
- d) Prototype

12) Un modelo de ciclo de vida estrechamente relacionado con el prototipado es:

- a) RUP.
- b) En espiral.
- c) Cascada.
- d) Por etapas.

13) GTK+ (GIMP Toolkit):

- a) Es una biblioteca de clases gráficas Java.
- b) Es una herramienta de prototipado.
- c) Es una plataforma gráfica para realizar wireframes.
- d) Es una biblioteca para crear interfaces graficas de usuario, construido encima de GDK (GIMP Drawing Kit).

14) Existen varios puntos de vista en una GUI:

- a) Modelo del usuario y del programador.
- b) Modelo de diseño y del programador.
- c) Modelo interno y externo.
- d) Modelo del usuario, del diseñador y del programador.

15) La técnica del prototipado:

- a) Siempre conviene utilizarla, pues mejora la comunicación.
- b) Convendrá utilizarla en función de las características del proyecto.
- c) No suele ser adecuado utilizarla debido al incremento de esfuerzo que introduce.
- d) Está desaconsejada por Métrica v3.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	b
2	a
3	c
4	b
5	a
6	a
7	c
8	c
9	a
10	c
11	c
12	b
13	d
14	d
15	b

BIBLIOGRAFÍA BÁSICA

1. Brooks, F. ***The Mythical Man-Month***. Ed. Addison-Wesley. 1975.
2. Connell, John L. y Shafer, Linda. ***Structured Rapid Prototyping: An Evolutionary Approach to Software Development***. Ed. Prentice Hall. Englewoods. NJ 1989.
3. Gordon, V.S, Bieman, J.M. ***Rapid Prototyping. Lessons Learned***. IEEE Software.
4. ***Metodología Métrica versión 3***. Ministerio de Administraciones Públicas. 1998.
5. Nogueira J. et al. ***Surfing the Edge of Chaos: Applications to Software Engineering***. Command and Control Research Technology Symposium, Naval Post Graduate School. Monterey. CA. Junio 2000.
http://www.dodccrp.org/2000CCRTS/cd/html/pdf_papers/Track_4/075.pdf
6. Pressman, Roger. S. ***Ingeniería del Software. Un Enfoque Práctico***. Ed. McGraw Hill. 4ª Ed. 1997. ISBN 84-481-1186-9
7. Pressman, Roger. ***Ingeniería del Software. Un Enfoque Práctico***. Ed. McGraw Hill. 6ª Ed. ISBN 970-105473-3
8. Sommerville, Ian. ***Ingeniería del Software***. Ed. Pearson-Addison-Wesley. 7ª Ed. 2005. ISBN 84-7829-074-5
9. Whitten et. al. ***Análisis de Sistemas y Métodos de Diseño***. Ed. Irwin. 3ª Ed. 1996. ISBN 84-8086-252-1
10. Diferenciando entre un sketch, mockup, wireframe y prototipo.
<http://www.otherwiseonline.net/diferencias-entre-sketch-mockup-wireframe-prototipo/>
11. Diseño de interfaz gráfica de usuario. María Claudia Albornoz.
http://sedici.unlp.edu.ar/bitstream/handle/10915/41578/Documento_completo.pdf?sequence=
12. Interfaz Gráfica de Usuario
<http://bibing.us.es/proyectos/abreproy/11300/fichero/PROYECTO%252FCapitulo3.pdf>
13. Diseño de interacción. Dr. Eduardo Huerta. <http://www.esdi.url.edu/content/pdf/disenio-de-interaccion.pdf>
14. Introducción al Diseño de Sistemas de Información. Diseño de Interfaz de Usuario.
15. La usabilidad en Ingeniería del software: definición y características. Walter Sánchez.
<http://redicces.org.sv/jspui/bitstream/10972/1937/1/2.%20La%20usabilidad%20en%20Ingenieria%20de%20Software-%20definicion%20y%20caracteristicas.pdf>
16. Interfaces de usuario: diseño de la visualización de la información como medio para mejorar la gestión del conocimiento y los resultados obtenidos por el usuario. Maria Jesús Fernández Ruiz.

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. 7 Principios de Diseño Universal o Diseño para todos. Fundación SIDAR.
<http://www.sidar.org/recur/desdi/usable/dudt.php>
2. Human Interface Guidelines de Apple: <https://developer.apple.com/design/>
3. Guía de Comunicación Digital para la Administración General del Estado
https://administracionelectronica.gob.es/pae/Home/pae_Documentacion/pae_Metodolog/pae_Guia_de_Comunicacion_Digital_para_la_Administracion_General_del_Estado.html#.WZSTSuntZdg
4. Estándares y guías. Ana Belén Martínez y Juan Manuel Cueva.
<http://aipo.es/libro/pdf/09Estand.pdf>